FESTIVO

A Project Report Submitted in Partial Fulfilment of the

Requirements for the Award of the Degree of

BACHELOR OF COMPUTER APPLICATIONS

By

Shajahan Basheer - 230021080724

Under the guidance of

Dr. Neetha Thomas

Assistant Professor

Department of Computer Science

SCCS
SANTHIGIRI COLLEGE
OF COMPUTER SCIENCES
Affiliated to M.G. University, Approved by AICTE and Accredited by NAAC

MARCH 2026

SANTHIGIRI COLLEGE
OF COMPUTER SCIENCES
Affiliated to M.G. University, Approved by AICTE and Accredited by NAAC

CERTIFICATE

*This is to certify that the report titled **Festivo** is a bonafide record of work done by **Shajahan Basheer (230021080724)** of **Santhigiri College of Computer Sciences** in partial fulfillment of the requirements of **Sixth** Semester of **Bachelor of Computer Applications** during the **year 2025**.*

Prof. Dr. Fr. Jose John CMI	**Mr. Gibin George**	**Dr. Neetha Thomas**
Principal	**HOD**	**Project Guide**

Internal Examiner **External Examnier**

DECLARATION

I **Shajahan Basheer** hereby declare that the project report, titled **"Festivo"** is a record of original work undertaken by us for the award of the degree of Bachelor of Computer Applications. I have completed this project under the guidance of Dr. Neetha Thomas, Assistant Professor, Department of Computer Science.

I also declare that this project has not been submitted for the award of any degree. I hereby confirm the originality of the work.

Name of the Student :

Register Number :

Signature :

Date :

Place :

ACKNOWLEDGEMENT

A project is not complete if one fails to acknowledge all who have been instrumental in the successful completion of the project. If words were to be the symbol of undiluted feelings and token of gratitude, then let the words play the heralding role of expressing our gratitude.

First of all, I thank the "**God Almighty**" for his immense grace and blessings in our life and at each stage of this project.

I express my sincere and profound gratitude to **Prof. Dr. Fr. Jose John CMI,** Principal, Santhigiri College of Computer Sciences, Vazhithala for providing all the facilities during the period of the project.

I extend my gratitude to **Mr. Gibin George** , Head of the Department of Computer Science, who is a constant source of inspiration and whose advice helped us to complete this project successfully.

I express my deep sense of gratitude to my internal project guide, **Dr Neetha Thomas,** Assistant Professor, Department of Computer Science, for her profound guidance for the successful completion of this project.

I am grateful to all the member of **SANTHISOFT TECHNOLOGIES** for their kind support and suggestion during all the development stages.

With great enthusiasm I express my gratitude to all the faculty members of Department of Computer Science for their timely help and support.

Finally, I express my deep appreciation to all my friends and family members for the moral support and encouragement they have given to complete this project successfully.

ABSTRACT

Festivo is a comprehensive online fest and event management platform developed to simplify, streamline, and modernize the way institutions plan, organize, publish, and manage their events. In many colleges, schools, training institutes, and organizations, event management is often handled manually through spreadsheets, paperwork, and offline coordination, which can lead to inefficiencies, miscommunication, and increased administrative workload. **Festivo** addresses these challenges by providing a centralized digital solution that integrates all aspects of event management into a single, user-friendly system.

The platform enables administrators and organizers to create and manage fests by defining event categories, setting up events, specifying rules, and handling participant registrations efficiently. It supports various event types such as technical competitions, cultural programs, workshops, seminars, and sports activities, ensuring flexibility for different institutional requirements. Organizers can monitor registrations in real time, manage participant data effectively, and track the overall progress of events with reduced effort.

For participants, **Festivo** offers a seamless and accessible experience where users can browse upcoming fests, explore event details, register for activities, and stay updated with schedules and announcements from anywhere at any time. The platform ensures clarity of information, reduces confusion, and enhances user engagement through timely notifications and updates. **Festivo** also incorporates role-based access for administrators, faculty or coordinators, and participants, ensuring proper control, security, and organization. By digitizing the event management process, the platform enhances transparency, minimizes manual errors, and reduces administrative effort while creating an efficient, scalable, and engaging environment for both organizers and participants.

LIST OF TABLES

Table Number	Table Name	Page Number
1.	Sign off Table	6
2.	tbl_login	23
3.	tbl_category	23
4.	tbl_plan	24
5.	tbl_institution	24
6.	tbl_subscription	25
7.	tbl_coordinators	25
8.	tbl_fests	26
9.	tbl_assign	26
10.	tbl_guest_contact	27
11.	tbl_competition	27
12.	tbl_participant	28
13.	tbl_competitionreg	29
14.	tbl_competition_group_details	29
15.	tbl_result	30
16.	tbl_payment	30
17.	Test Case	73

LIST OF FIGURES

Figure Number	Figure Name	Page Number
1.1	Zeroth Level DFD	33
1.2	First Level DFD	33
1.3	Second Level DFD for Registration	34
1.4	Second Level DFD for Core Authentication	35
1.5	Second Level DFD for Payments	36
1.6	Second Level DFD for Activities	36
1.7	Second Level DFD for Reports	38
3.8	Institution Registration Form	41
3.9	View All Users Page	43
4.1	Registration Form	45
5.1	Unit Testing	69
5.2	Unit Testing Result	69
5.3	Integration Testing	70
5.4	Integration Testing Result	71
5.5	Registration Field Validation	72
5.6	Registration Email Validation	72
8.1	Guest Page	78
8.2	Institution Registration	78
8.3	Login Page	79
8.4	Fest View page	79
8.5	Fest Details Page	80
8.6	Competition registration page	81
8.7	Institution dashboard	81
8.8	Fest creating page	82
8.9	Fest view page on institution	82
8.10	Fest details page	83
8.11	Coordinator assign page	83
8.12	Coordinator registration page	84

8.13	Coordinator view page	84
8.14	View competitions page	85
8.15	Competition details page	85
8.16	Participant view page	86
8.17	Student report page	86
8.18	Revenue report page	87
8.19	Prize report page	87
8.20	Coordinator dashboard	88
8.21	Coordinator participant view page	88
8.22	Coordinator attendance mark page	89
8.23	Coordinator manual/spot registration page	89
8.24	Admin category registration	90
8.25	Admin category view page	90
8.26	Admin view all user page	91
8.27	Admin overall report page	91

ABBREVIATIONS

IDE	Integrated Development Environment
CPU	Central Processing Unit
DBMS	Data Base Management System
RDBMS	Relational Data Base Management System
NF	Normal Forms
PK	Primary Key
FK	Foreign Key
DFD	Data Flow Diagram

TABLE OF CONTENTS

Acknowledgement **IV**

Abstract **V**

List of Tables **VI**

List of Figures **VII**

Abbreviation **VIII**

1. Introduction 1
   - 1.1 Background and Motivation 1
   - 1.2 The Proposed System 1
   - 1.3 Project Scope 2
     - 1.3.1 Limitations of Existing System 2
     - 1.3.2 Advantages of Proposed System 3
2. System Analysis 4
   - 2.1 Introduction 4
   - 2.2 Stake Holders of this Project 4
     - 2.2.1 Admin 4
     - 2.2.2 Institution 4
     - 2.2.3 Coordinator 5
     - 2.2.4 Participant
   - 2.3 Software Requirements Specification 5
     - 2.3.1 Admin 5
     - 2.3.2 Institution 6
     - 2.3.3 Coordinator 6
     - 2.3.4 Participant
   - 2.4 Feasibility Study 6
   - 2.5 Software Development Lifecycle Model 9
   - 2.6 Hardware and Software Requirements 10
     - 2.6.1 Software Specifications 10
       - 2.6.1.1 ANGULAR,BOOTSTRAP,EXPRESS 10
       - 2.6.1.2 MySQL 11
       - 2.6.1.3 Visual Studio 13
       - 2.6.1.4 Windows 11 14
       - 2.6.1.5 Microsoft Word 15
     - 2.6.2 Hardware Requirements 15

3. System Design 17
3.1 System Architecture 17
3.2 Module Design 18
3.3 Database Design 20
3.3.1 Normalization 20
3.3.2 Table Structure 22
3.4 Data Flow Diagram 31
3.4.1 Introduction to Data Flow Diagrams 31
3.4.2 Data Flow Diagram 33
3.5 Interface Design 40
3.5.1 User Interface Screen Design 40
3.5.2 Output Design 42
4. Implementation 44
4.1 Coding Standards 44
4.2 Sample Code 45
5. Testing 67
5.1 Test Cases 67
5.1.1 Unit Testing 68
5.1.2 Integration Testing 70
5.1.3 Validation Testing 71
5.1.4 User Acceptance Testing 72
5.2 Test Case Documents 73
6. Conclusion 76
7. References 77
8. Appendix 78
8.1 Screenshots 78

1. INTRODUCTION

1.1 BACKGROUND AND MOTIVATION

Festivals and events are an essential part of educational institutions and organizations, including colleges, schools, and other institutions. These events may include cultural festivals, technical fests, academic competitions, workshops, exhibitions, and sports events. Organizing such events involves multiple activities such as event planning, participant registration, schedule management, coordination among organizers, and communication with participants.

In many institutions, fest management is still handled using traditional manual methods such as paper-based registrations, spreadsheets, notice boards, and informal communication channels. These approaches are inefficient, time-consuming, and prone to errors such as data duplication, loss of information, and lack of proper coordination. As the number of events and participants increases, managing the entire process manually becomes difficult and unreliable.

The motivation behind developing **Festivo – Fest Management System** is to provide a centralized, web-based platform that can be used by colleges, schools, and other institutions to efficiently host and manage festivals and events. The system aims to simplify event management by automating registrations, organizing participant data, managing schedules, and providing timely updates to users. **Festivo** reduces manual effort, improves accuracy, and enhances communication between organizers and participants. Additionally, the project serves as a practical implementation of modern web technologies, helping to address real-world event management challenges in an efficient and scalable manner.

1.2 THE PROPOSED SYSTEM

Festivo – Fest Management System is a web-based application developed to manage and organize festivals and events conducted by colleges, schools, and other institutions using a centralized digital platform. The system automates essential fest activities such as event creation, participant registration, schedule management, and communication.

- **Admin**, who manages the overall system, institutions, and user access
- **Institution**, which registers and hosts festivals and manages institutional details.
- **Coordinator**, who organizes events, manages schedules, and monitors participant

registrations

- **Participant**, who views events, registers online, and receives updates

Festivo replaces traditional manual methods with a secure, database-driven system, reducing paperwork and human errors while improving data accuracy. The system enhances coordination among stakeholders, ensures easy access to information, and improves the efficiency and transparency of fest management. Being web-based, Festivo is accessible from anywhere and can be easily scaled to support multiple institutions and events.

PROJECT SCOPE

1.2.1 Limitations of Existing System

1. **Manual Registration Process** – The existing system mainly depends on paper forms or offline registration methods. This process is slow, requires more manpower, and becomes difficult to manage when the number of participants increases.
2. **High Possibility of Errors** – Since data is entered and managed manually, there is a high chance of errors such as duplicate registrations, incorrect participant details, and loss of important information.
3. **Lack of Centralized Management** – Fest-related data such as event details, participant information, and schedules are stored in different places. This lack of a centralized system leads to poor coordination among administrators, institutions, coordinators, and participants.
4. **Inefficient Communication** – Information about events, schedule changes, and announcements is usually shared through notice boards or word-of-mouth. This often results in delayed or missed communication to participants.
5. **Difficulty in Tracking and Monitoring** – Tracking participant registrations, event participation, and overall fest progress is difficult using manual records or spreadsheets, especially for large-scale events.
6. **Time-Consuming Report Generation** – Preparing reports such as participant lists and event summaries manually requires additional time and effort and may not always be accurate.
7. **Poor Scalability** – The existing system is not suitable for handling large or multi-institution festivals, as managing increased data and users becomes inefficient and unreliable.

1.2.2 Advantages of Proposed System

1. **Centralized Fest Management** – The proposed system provides a single centralized platform to manage all fest-related activities, including events, participants, schedules, and announcements, ensuring better coordination among stakeholders.
2. **Online Registration System** – Participants can register for events online, which eliminates paperwork and reduces manual effort for coordinators and institutions.
3. **Improved Accuracy and Data Integrity** – Since data is stored in a structured database, the system minimizes errors such as duplicate entries and incorrect information, ensuring accurate and reliable data management.
4. **Role-Based Access Control** – Different stakeholders such as Admin, Institution, Coordinator, and Participant have specific access rights, which improves security and ensures proper management of responsibilities.
5. **Time and Cost Efficiency** – Automation of fest activities reduces the time and manpower required to manage events, thereby lowering operational costs.
6. **Easy Tracking and Monitoring** – Coordinators and institutions can easily track participant registrations, event status, and fest progress through the system.
7. **Scalability and Flexibility** – The system can support multiple institutions and large-scale events, making it suitable for various types of festivals.
8. **User-Friendly Interface** – Festivo is designed with a simple and intuitive interface, making it easy to use for all types of users, including non-technical users.

2. SYSTEM ANALYSIS

2.1 INTRODUCTION

Software Engineering is the analysis, design, construction, verification and management of technical or social entities. To engineer software accurately, a software engineering process must be defined. System analysis is a detailed study of the various operations performed by the system and their relationship within and module of the system. It is a structured method for solving the problems related to the development of a new system. The detailed investigation of the present system is the focal point of system analysis. This phase involves the study of parent system and identification of system objectives. Information has to be collected from all people who are affected by or who use the system. During analysis, data are collected on the variable files, decision point and transactions handled by the present system. The main aim of system is to provide the efficient and user-friendly automation. So the system analysis process should be performed with extreme precision, so that an accurate picture of existing system, its disadvantages and the requirements of the new system can be obtained.

2.2 STAKEHOLDERS IN THIS PROJECT

2.2.1 ADMIN

The Admin is the highest-level stakeholder in the Festivo system and is responsible for overall system management. The Admin controls user access, manages registered institutions, and ensures the smooth functioning of the system. The Admin also monitors system activities, maintains data integrity, and handles system-level configurations and security.

2.2.2 Institution

The Institution stakeholder represents colleges, schools, or other organizations that host festivals using the Festivo system. Institutions can register on the platform, create and manage festivals, and oversee event activities under their organization. This stakeholder ensures that fest information, coordinators, and events are properly managed within the institution.

2.2.3 Coordinator

The Coordinator is responsible for managing individual events within a festival. Coordinators create events, manage event schedules, and monitor participant registrations. They act as the link between the institution and participants, ensuring smooth execution of events and timely updates.

2.2.4 Participant

The Participant is the end user of the system who registers to take part in events. Participants can view available festivals and events, register online, and receive updates related to event schedules and announcements. The system provides participants with easy access to fest information, improving their overall participation experience.

2.3 SOFTWARE REQUIREMENT SPECIFICATION

2.3.1 ADMIN

1. Admin should have the facility to login using username and password.
2. Admin can manage registered institutions in the system.
3. Admin can monitor all festivals and events created by institutions.
4. Admin can view overall participant and event details.
5. Admin can generate reports related to system usage and activity.
6. Admin should have the facility to Logout.

2.3.2 Institution

1. Institution should have the facility to login using a valid username and password.
2. Institution can create and manage festivals hosted by the organization.
3. Institution can add, manage, and remove coordinators for their festivals.
4. Institution can view and monitor all events under their festivals.
5. Institution can view participant registrations for each event.
6. Institution can publish fest-related updates and schedules.
7. Institution can generate reports related to festivals, events, and participants.
8. Institution should have the facility to logout.

2.3.3 Coordinator

1. Coordinator should have the facility to login using a valid username and password.
2. Coordinator can view participant details of the events assigned to them..
3. Coordinator can mark attendance for participants in their events.
4. Coordinator can add results for competitions or events under their responsibility.
5. Coordinator can view schedules and updates related to their events.
6. Coordinator should have the facility to logout.

2.3.4 Particiapant

1. Participant should have the facility to register on the platform.
2. Participant should have the facility to login using a valid username and password.
3. Participant can view available festivals and events.
4. Participant can register for events online.
5. Participant can view event schedules and updates.
6. Participant can view their registered events and participation status.
7. Participant should have the facility to logout.

Table 2.1 Sign off table

SI.No.	Name & Designation	Date	Accepted (Yes/No)
1	Dr. Neetha Thomas Assistant Professor Santhigiri College of Computer Sciences		
2	Shajahan Basheer Developer		

2.4 FEASIBILITY STUDY

Feasibility is defined as the practical extent to which a project can be performed successfully. To evaluate feasibility, a feasibility study is performed, which determines whether the solution considered to accomplish the requirements is practical and workable in the software. Information such as resource availability, cost estimation for software development, benefits of the software to the organization after it is developed and cost to be incurred on its maintenance are considered during the feasibility study.

The objective of the feasibility study is to establish the reasons for developing the software that is acceptable to users, adaptable to change and conformable to established standards.

Various other objectives of feasibility study are listed below.

- To analyze whether the software will meet organizational requirements.
- To determine whether the software can be implemented using the current technology and within the specified budget and schedule.
- To determine whether the software can be integrated with other existing software. When our project guide as well as our client Dr. Neetha Thomas told us regarding the mini project and about Word to the Wise for getting the desired product developed, it comes up with rough idea about what all functions the software must perform and which all features are expected from the software.

Referencing to this information, we do a studies and discussions about whether the desired system and its functionality are feasible to develop and the output of this phase is a feasibility study report that should contained adequate comments and recommendations.Various types of feasibility that we checked include technical feasibility, operational feasibility, and economic feasibility.

Technical Feasibility

Technical feasibility assesses the current resources (such as hardware and software) and technology, which are required to accomplish user requirements in the software within the allocated time and budget. For this, the software development team ascertains whether the current resources and technology can be upgraded or added in the software to accomplish specified user requirements. Technical feasibility also performs the following tasks.

- Analyses the technical skills and capabilities of the software development team members.
- Determines whether the relevant technology is stable and established.
- Ascertains that the technology chosen for software development has a large number of users so that they can be consulted when problems arise or improvements are required.

From our perspective there are two languages Angular, HTML and database MySQL which are used to develop this web-based applications. PHP is used in the front end and MySQL is used in the back end. The Word to the Wise is web based and thus can be accessed through any browsers. As we are using these latest technologies which are currently trending and used by a number of developers across the globe, we can say that our project is technically feasible.

Operational Feasibility

Operational feasibility assesses the extent to which the required software performs a series of steps to solve business problems and user requirements. This feasibility is dependent on human resources (software development team) and involves visualizing whether the software will operate after it is developed and be operative once it is installed. Operational feasibility also performs the following tasks.

- Determines whether the problems anticipated in user requirements are of high priority.
- Determines whether the solution suggested by the software development team is acceptable.
- Analyses whether users will adapt to a new software.
- Determines whether the organization is satisfied by the alternative solutions proposed by the software development team.

We found that our project will be satisfied for the client since we were discussing every detail about the software with the client at every step. The most important part of operational feasibility study is the input from client. So the software is built completely according to the requirements of the client. We have used the current industry standards for the software. Hence, we can say that this software is operationally feasible.

Economic Feasibility

Economic feasibility determines whether the required software is capable of generating financial gains for an organization. It involves the cost incurred on the software development team, estimated cost of hardware and software, cost of performing feasibility study, and so on. For this, it is essential to consider expenses made on purchases (such as hardware purchase) and activities required to carry out software development. In addition, it is necessary to consider the benefits that can be achieved by developing the software. Software is said to be economically feasible if it focuses on the issues listed below.

- Cost incurred on software development to produce long-term gains for an organization.
- Cost required to conduct full software investigation (such as requirements elicitation and requirements analysis).

- Cost of hardware, software, development team, and training.

It is estimated that our project is economically feasible as development cost is very minimal since the tools and technologies used are available online. It's a group student project so there are no personnel costs. Development time is well planned and will not affect other operations and activities of the individuals. Once the system has been developed, the companies purchasing the system will be providing with a manual for training purposes. There is no need to purchase new hardware since the existing computers can still be used to implement the new system.

2.5 SOFTWARE DEVELOPMENT LIFECYCLE MODEL

One of the basic notions of the software development process is SDLC models which stand for Software Development Life Cycle models. SDLC – is a continuous process, which starts from the moment, when it's made a decision to launch the project,and it ends at the moment of its full remove from the exploitation. Software development lifecycle (SDLC) is a framework that defines the steps involved in the development of software. It covers the detailed plan for building, deploying and maintaining the software. SDLC defines the complete cycle of development i.e. all the tasks involved in gathering a requirement for the maintenance of a Product. Some of the common SDLC models are Waterfall Model, V-Shaped Model, Prototype Model, Spiral Model, Iterative Incremental Model, Big Bang Model, Agile Model. We used Agile Model for our Project.

Agile Model

Agile Model is a combination of the Iterative and incremental model. This model focuses more on flexibility while developing a product rather than on the requirement. In the agile methodology after every development iteration, the client is able to see the result and understand if he is satisfied with it or he is not. In Agile, a product is broken into small incremental builds. It is not developed as a complete product in one go. At the end of each sprint, the project guide verifies the product and after his approval, it is finalized. Client feedback is taken for improvement and his suggestions and enhancement are worked on in the next sprint. Testing is done in each sprint to minimize the risk of any failures.

Advantages of Agile Model:

- It allows more flexibility to adapt to the changes.
- The new feature can be added easily.

- Customer satisfaction as the feedback and suggestions are taken at every stage.
- Risks are minimized thanks to the flexible change process.

Disadvantages:

- Lack of documentation.
- If a customer is not clear about how exactly they want the product to be, then the project would fail.
- With all the corrections and changes there is possibility that the project will exceed expected time.

2.6 HARDWARE AND SOFTWARE REQUIREMENTS

2.6.1 SOFTWARE SPECIFICATIONS

This project is built upon the latest technology software.

Front end	:	HTML, CSS
Development tool	:	Angular, Bootstrap, Express
Database	:	MySQL
Web server	:	Internet Information Server (IIS)
Operating System	:	Windows 11

2.6.1.1 ANGULAR, BOOTSTRAP, EXPRESS

Angular is a popular open-source web application framework developed and maintained by Google. It's used to build dynamic single-page web applications. Angular provides a structured framework for building web apps using TypeScript, a superset of JavaScript. Key features of Angular include its use of components as building blocks for UI, a powerful dependency injection system, two-way data binding, routing, and extensive support for building responsive and scalable applications. Angular's modular and comprehensive nature, along with its robust ecosystem, makes it suitable for creating complex and maintainable web applications.

Bootstrap is a widely-used, free, and open-source CSS framework developed by Twitter. It provides a collection of tools and pre-built components (such as buttons, forms, navigation bars, etc.) to help developers create responsive and visually appealing websites or web applications.

Express is a minimal and flexible web application framework for Node.js. It provides a robust set of features to develop web applications and APIs quickly and efficiently. Express simplifies the process of building web servers in Node.js by offering a simple and minimalistic framework with a rich set of HTTP utility methods and middleware. Angular encourages the use of components as the basic building blocks for user interfaces (UI). This component-based architecture promotes code reusability and maintainability. Emphasizing responsive design, Bootstrap equips developers with tools for crafting visually appealing interfaces that adapt seamlessly across diverse devices. Its customization capabilities, comprehensive documentation, and vibrant community support make it a preferred choice for developers seeking efficient and standardized web development solutions. Complementing these front-end technologies, Express, with its minimalist approach, facilitates rapid API and web server development. It offers a robust routing system, middleware support for tasks like authentication and error handling, and integration with various template engines. Its middleware-based architecture promotes modularity and extensibility, while its active community and rich ecosystem further bolster its appeal among Node.js developers aiming for efficient web application development. Together, Angular, Bootstrap, and Express form a formidable trio, enabling developers to create sophisticated, responsive, and scalable web applications with ease and efficiency.

2.6.1.2 MySQL

MySQL is the world's most popular open-source database software, with over 100 million copies of its software downloaded or distributed throughout its history. With its superior speed, reliability, and ease of use, MySQL has become the preferred choice for Web, Web 2.0, SaaS, ISV, Telecom companies and forward-thinking corporate IT Managers because it eliminates the major problems associated with downtime, maintenance and administration for modern, online applications.

MySQL is a key part of LAMP (Linux, Apache, MySQL, PHP / Perl / Python), the fast-growing open-source enterprise software stack. More and more companies are using LAMP as an alternative to expensive proprietary software stacks because of its lower cost and freedom from platform lock-in.

MySQL was originally founded and developed in Sweden by two Swedes and a Finn: David Axmark, Allan Larsson and Michael "Monty" Widenius, who had worked together since the 1980's. MySQL, the most popular Open-Source SQL database management

system, is developed, distributed, and supported by Oracle Corporation.

MySQL is a database management system. A database is a structured collection of data. It may be anything from a simple shopping list to a picture gallery or the vast amounts of information in a corporate network. To add, access, and process data stored in a computer database, you need a database management system such as MySQL Server. Since computers are very good at handling large amounts of data, database management systems play a central role in computing, as standalone utilities, or as parts of other applications.

MySQL databases are relational. A relational database stores data in separate tables rather than putting all the data in one big storeroom. The database structures are organized into physical files optimized for speed. The logical model, with objects such as databases, tables, views, rows, and columns, offers a flexible programming environment. You set up rules governing the relationships between different data fields, such as one-to one, one-tomany, unique, required or optional, and ―pointers‖ between different tables. The database enforces these rules, so that with a well-designed database, your application never sees inconsistent, duplicate, orphan, out-of-date, or missing data.

The SQL part of ―MySQL‖ stands for ―Structured Query Language‖. SQL is the most common standardized language used to access databases. Depending on your programming environment, you might enter SQL directly ,embed SQL statements into code written in another language, or use a language-specific API that hides the SQL syntax.

SQL is defined by the ANSI/ISO SQL Standard. The SQL standard has been evolving since 1986 and several versions exist. In this manual, ―SQL-92‖ refers to the standard released in 1992, ―SQL:1999‖ refers to the standard released in 1999, and ―SQL:2003‖ refers to the current version of the standard.

We use the phrase ―the SQL standard‖ to mean the current version of the SQL Standard at any time. MySQL software is Open Source. Open-Source means that it is possible for anyone to use and modify the software. Anybody can download the MySQL software from the Internet and use it without paying anything. If you wish, you may study the source code and change it to suit your needs.

The MySQL software use the GPL (GNU General Public License), http://www.fsf.org/licenses/, to define what you may and may not do with the software in different situations.

The MySQL Database Server is very fast, reliable, scalable, and easy to use. If that is what you are looking for, you should give it a try. MySQL Server can run comfortably on a desktop or laptop, alongside your other applications, web servers, and so on, requiring little or no attention. If you dedicate an entire machine to MySQL, you can adjust the settings to take advantage of all the memory, CPU power, and I/O capacity available. MySQL can also scale up to clusters of machines, networked together.

The MySQL Database Software is a client/server system that consists of a multi- threaded SQL server that supports different backends, several different client programs and libraries, administrative tools, and a wide range of application programming interfaces (APIs). We also provide MySQL Server as an embedded multi-threaded library that you can link into your application to get a smaller, faster, easier-to-manage standalone product.

A large amount of contributed MySQL software is available. MySQL Server has a practical set of features developed in close cooperation with our users. It is very likely that your favorite application or language supports the MySQL Database Server.

2.6.1.3 VISUAL STUDIO

Visual Studio Code is a source-code editor that can be used with a variety of programming languages, including C, C#, C++, FORTRAN, Go, Java, JavaScript, Node.js, Python, Rust, and Julia. It is based on the Electron framework, which is used to develop Node.js web applications that run on the Blink layout engine. Visual Studio Code employs the same editor component (code named "Monaco") used in Azure DevOps.

Out of the box, Visual Studio Code includes basic support for most common programming languages. This basic support includes syntax highlighting, bracket matching, code folding, and configurable snippets. Visual Studio Code also ships with Intelligent for JavaScript, Type Script, JSON, CSS, and HTML. Support for additional languages can be provided by freely available extensions on the VS Code Marketplace. Instead of a project system, it allows users to open one or more directories, which can then be saved in work spaces for future reuse. This allows it to operate as a language-agnostic code editor for any language. It supports many programming languages and a set of features that differs per language. Unwanted files and folders can be excluded from the project tree via the settings. Many Visual Studio Code features are not exposed through menus or the user interface but can be accessed via the command palette.

2.6.1.4 WINDOWS 11

An operating system can be viewed as a scheduler, where it has resources for which it has charge. Resources include CPU, memory, I/O device and disk space. In another view, the operating system is a new machine. The third view is that operating system is a multiplexer which allows sharing of resources provides protection from interference and provides a level of cooperation between users. This project is developed using Windows 11 as the operating system and supports its latest versions. Windows 11 is the latest installment in Microsoft's Windows NT family of operating systems, succeeding Windows 10. It was officially announced on June 24, 2021, and released to manufacturing on October 5, 2021, with general availability starting on October 5, 2021. Windows 11 introduces a refreshed user interface with centered Start Menu and taskbar icons, enhanced snap layouts, and virtual desktops. It also integrates Microsoft Teams directly into the operating system, providing seamless communication and collaboration capabilities. Other new features include support for running Android apps through the Amazon Appstore, a revamped Microsoft Store with a broader range of applications, improved gaming performance through Auto HDR and DirectStorage, and enhanced security features such as Windows Hello for faster, more secure login options. Initial reviews of Windows 11 have been generally positive, praising its design changes, performance improvements, and new features. Critics have noted concerns over hardware requirements and compatibility issues, particularly with older hardware, as well as some controversial decisions such as the requirement for a Microsoft account and internet connection during setup. Despite these criticisms, Windows 11 represents Microsoft's continued evolution of its operating system, aiming to enhance productivity, security, and user experience across a range of devices.

2.6.1.5 MICROSOFT WORD

Microsoft Word (or simply Word) is a word processor developed by Microsoft. It was first released on October 25, 1983 under the name Multi-Tool Word for Xenix systems. Subsequent versions were later written for several other platforms including IBM PCs running DOS (1983), Apple Macintosh running the Classic Mac OS (1985), AT&T Unix PC (1985), Atari ST (1988), OS/2 (1989), Microsoft Windows (1989), SCO Unix (1994), and macOS (formerly OS X; 2001).

Commercial versions of Word are licensed as a standalone product or as a component of Microsoft Office, Windows RT or the discontinued Microsoft Works suite. Unlike most MS-DOS programs at the time, Microsoft Word was designed to be used with a mouse. Advertisements depicted the Microsoft Mouse, and described Word as a WYSIWYG, windowed word processor with the ability to undo and display bold, italic, and underlined text, although it could not render fonts. It was not initially popular, since its user interface was different from the leading word processor at the time, WordStar. However, Microsoft steadily improved the product, releasing versions 2.0 through 5.0 over the next six years. In 1985, Microsoft ported Word to the classic Mac OS. This was made easier by Word for DOS having been designed for use with high-resolution displays and laser printers, even though none were yet available to the general public. Following the precedents of LisaWrite and MacWrite, Word for Mac OS added true WYSIWYG features. It fulfilled a need for a word processor that was more capable than MacWrite. After its release, Word for Mac OS's sales were higher than its MS-DOS counterpart for at least four years.

2.6.2 HARDWARE REQUIREMENTS

The selection of hardware configuring is a very task related to the software development, particularly inefficient RAM may affect adversely on the speed and corresponding on the efficiency of the entire system. The processor should be powerful to handle all the operations. The hard disk should have the sufficient to solve the database and the application.

Hardware used for development:

CPU	:	Intel i5 Processer
Memory	:	4 GB
Cache	:	6 MB
Hard Disk	:	1 TB
Monitor	:	15.6” Monitor
Keyboard	:	Standard108 keys Enhanced Keyboard
Mouse	:	Optical Mouse

Minimum Hardware Required for Implementation:

CPU	:	Pentium IV Processor
Memory	:	256MB Above
Cache	:	512 KB Above
Hard Disk	:	20 GB Above
Monitor	:	Any
Keyboard	:	Any
Mouse	:	Any

3. SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

A system architecture or system's architecture is the conceptual model that defines the structure, behavior, and more views of a system. An architecture description is a formal description and representation of a system, organized in a way that supports reasoning about the structures of the system,

System architecture can comprise system components, the externally visible properties of those components, the relationships (e.g. the behaviour) between them. It can provide a plan from which products can be procured, and systems developed, that will work together to implement the overall system. There have been efforts to formalize languages to describe system architecture; collectively these are called architecture description languages (ADLs).

The system architecture can best be thought of as a set of representations of an existing (or to be created) system. It is used to convey the informational content of the elements comprising a system, the relationships among those elements, and the rules governing those relationships. The architectural components and set of relationships between these components that architecture describes may consist of hardware, software, documentation, facilities, manual procedures, or roles played by organizations or people. System architecture is primarily concerned with the internal interfaces among the system's components or subsystems, and the interface between the system and its external environment, especially the user.

The structural design reduces complexity, facilitates change and result in easier implementation by encouraging parallel development of different parts of the system. The procedural design transforms structural elements of program architecture into a procedural description of software components. The architectural design considers architecture as the most important functional requirement. The system is based on the three-tier architecture.

The first level is the user interface (presentation logic), which displays controls, receives and validates user input. The second level is the business layer (business logic) where the application specific logic takes place. The third level is the data layer where the application information is stored in files or database. It contains logic about to retrieve

and update data. The important feature about the three-tier design is that information only travels from one level to an adjacent level.

3.2 MODULE DESIGN

Modular programming is a software design technique that emphasizes separating the functionality of a program into independent, interchangeable modules, such that each contains everything necessary to execute only one aspect of the desired functionality. Conceptually, modules represent a separation of concerns, and improve maintainability by enforcing logical boundaries between components.

Different modules of this project include.

1. Registration

The Registration module allows all stakeholders to create accounts in the system. Admin can register institutions, institutions can register festivals, and participants can register to take part in events. During registration, users provide basic details such as name, email, phone number, username, and password, which are securely stored in the database. This module ensures that only authorized users can access the system and participate in the festival management process.

2. User Authentication

The User Authentication module ensures secure access for all stakeholders. Admin, institutions, coordinators, and participants can log in using their username and password. The module allows users to change passwords or reset forgotten passwords. It also restricts access based on roles, ensuring that each stakeholder can perform only the actions permitted for their role in the system.

3. Fest Management

The Fest Management module enables institutions to create and manage festivals and associated events. Institutions can define event details such as name, schedule, and participant limits, and assign coordinators to each event. Participants can view festival and event information and register for events online. This module provides a structured and organized approach to handling all festival-related activities, ensuring proper scheduling and coordination..

4. Coordinator Management

Coordinator Management allows institutions to manage coordinators for their festivals. Institutions can add, update, or remove coordinators as needed. Coordinators are assigned to specific events, where they can view participant details, mark attendance, and enter results. This module ensures clear role-based responsibilities and improves the management of events under each festival.

5. Attendance And Results Management

This module allows coordinators to track attendance and record results for their assigned events. Coordinators can mark participant attendance, update event outcomes, and maintain accurate records. Participants can view their attendance status and results through their accounts. The module ensures transparency and accurate tracking of event performance and participation.

6. Payment

The Payment module manages registration or event fees for participants, if applicable. Participants can pay fees online, and institutions and admins can monitor received payments. The system can automatically calculate fees based on event type or participation, and generate payment reports. This module provides a smooth and secure method for handling all financial transactions related to festivals and events.

7. Reports

The Reports module generates analytical and statistical reports for all stakeholders. Admin can generate system-wide reports about institutions, festivals, events, and participant activity. Institutions can view festival-specific reports including participant registrations and event attendance. Coordinators can generate event-specific reports such as attendance and results. Reports can be displayed as tables, charts, PDFs, or Excel files to help stakeholders make informed decisions and track festival performance.

3.3 DATABASE DESIGN

A database is a collection of interrelated data stored with minimum redundancy to serve many users quickly and efficiently. The general objective is to make information access easy, quick, inexpensive and flexible for the users. The general theme behind a database is to integrate all information. Database design is recognized as a standard of management information system and is available virtually for every computer system. In database design several specific objectives are considered:

- Ease of learning and use
- Controlled redundancy
- Data independence
- More information at low cost
- Accuracy and integrity
- Recovery from failure
- Privacy and security
- Performance

A database is an integrated collection of data and provides centralized access to the data. Usually, the centralized data managing the software is called RDBMS. The main significant difference between RDBMS and other DBMS is the separation of data as seen by the program and data has in direct access to stores device. This is the difference between logical and physical data.

3.3.1 Normalization

Designing a database is complete task and the normalization theory is a useful aid in the design process. The process of normalization is concerned with transformation of conceptual schema into computer representation form. There will be need for most databases to grow by adding new attributes and new relations. Tuples will be added and deleted. Information stored may undergo updating also. New association may also be added. In such situations the performance of a database is entirely depend upon its design. A bad database design may lead to certain undesirable things like:

- Repetition of information
- Inability to represent certain information
- Loss of information

To minimize these anomalies, Normalization may be used. If the database is in a normalized form, the data can be growing without, in most cases, forcing the rewriting

application programs. This is important because of the excessive and growing cost of maintaining an organization's application programs and its data from the disrupting effects of database growth. As the quality of application programs increases, the cost of maintaining the without normalization will rise to prohibitive levels. A normalized database can also encompass many related activities of an organization thereby minimizing the need for rewriting the applications of programs. Thus, normalization helps one attain a good database design and there by ensures continued efficiency of database.

Normalization theory is built around the concept of normal forms. A relation is said to be in normal form if it satisfies a certain specified set of constraints. For example, a relation is said to be in first normal form (1NF) if it satisfies the constraint that it contains atomic values only. Thus, every normalized relation is in 1NF.Numerous normal forms have been defined. Codd defined the first three normal forms.

All normalized relations are in 1NF, some 1NF relations are also in 2NF and some 2NF relations are also in 3NF.2NF relations are more desirable than 1Nf and 3NF are more desirable than 2NF. That is, the database designer should prefer 3NF than 1NF or 2NF.Normalization procedure states that a relation that is in some given normal form can be converted into a set of relations in a more desirable form. We can define this procedure as the successive reduction of a given collection of relations to some more desirable form. This procedure is reversible. That is, it is always possible to take the output from the procedure and convert them back into input. In this process, no information is lost. So it is also called "no loss decomposition".

First Normal Form

A relation is in first normal form (1NF) if and all its attributes are based on single domain. The objective of normalizing a table is to remove its repeating groups and ensure that all entries of the resulting table have at most single value.

Second Normal Form

A table is said to be second Normal Form (2NF), when it is in 1NF and every attribute in record is functionally dependent upon the whole key, and not just a part of the key.

Third Normal Form

A table is in third Normal Form (3NF), when it is in 2NF and every non-key attribute is functionally dependent on just the primary key.

3.3.2 Table Structure

Table is a collection of complete details about a particular subject. These data are saved in rows and Columns. The data of each Row are different units. Hence, rows are called RECORDS and Columns of each row are called FIELDS.

Data is stored in tables, which is available in the backend the items and data, which are entered in the input, form id directly stored in this table using linking of database. We can link more than one table to input forms. We can collect the details from the different tables to display on the output.

There are mainly 15 tables in the project. They are,

1. tbl_login
2. tbl_category
3. tbl_plan
4. tbl_institution
5. tbl_subscription
6. tbl_coordinators
7. tbl_fests
8. tbl_assign
9. tbl_guest_contact
10. tbl_competition
11. tbl_participant
12. tbl_competitionreg
13. tbl_competition_group_details
14. tbl_result
15. tbl_payment

1. **Table Name: tbl_login**

Description: This table is used to store login information

Table 3.1 tbl_login

Field Name	**Data Type**	**Constraint**	**Description**
login_id	int	Primary Key	This field is used to uniquely identify logged user, and the values are automatically generated
username	varchar (50)	Not null	This field stores the username of the user.
password	varchar (50)	Not null	This field stores the password of the user.
role	varchar(50)	Not null	This field represent the role
status	Varchar(20)	Not null	This field represent the status of login

2. Table Name: **tbl_category**

Description: This table is used to store the categories of fest.

Table 3.2 tbl_category

Field Name	**Data Type**	**Constraint**	**Description**
category_id	int	Primary Key	This field is used to uniquely identify district, and the values are automatically generated
category_name	varchar (100)	Not null	This field stores the name of the district.
category_description	varchar (500)	Not null	This field stores the description of the category
category_image	varchar (100)	Null	This field is used to store the name of the category image.

3. Table Name: **tbl_plan**

Description: This table is used to store the plans for the fest.

Table 3.3 tbl_plan

Field Name	Data Type	Constraint	Description
plan_id	int	Primary Key	This field is used to uniquely identify plans, and the values are automatically generated
plan_name	varchar (50)	Not null	This field stores the name of the plan.
days	int	Not null	This field is used to specify the number of days of the plan
amount	varchar (10)	Not null	This field is used to store the amount of the plan
description	varchar (500)	Not null	This field is used to store the description of the plan

4. Table Name: **tbl_institution**

Description: This table is used to store the institution information.

Table 3.4 tbl_institution

Field Name	Data Type	Constraint	Description
institution_id	int	Primary Key	This field is used to uniquely identify Institutions, and the values are automatically generated.
institution_name	varchar(50)	Not null	This field stores the name of the institution.
institution_address	Varchar(50)	Not null	This field is used to store address of the institution.
institution_mail	varchar(50)	Not null	This field is used to store the e-mail of the institution
institution_contactno	varchar(15)	Not null	This field is used to store the contact number of the institution
institution_image	varchar(100)	Null	This field is used to store the image of the institution
login_id	int	Foreign Key	This field is a reference to the table tbl_login
websiteaddress	varchar(200)	Null	This field store the website address of the institution

5. Table Name: **tbl_subscription**

Description: This table is used to store subscription details by the institution.

Table 3.5 tbl_subscription

Field Name	**Data Type**	**Constraint**	**Description**
subscriptionid	int	Primary Key	This field is used to uniquely identify subscription by each institution, and the values are automatically generated.
planid	varchar(50)	Foreign key	This field is a reference to table tbl_plan.
institutionid	varchar(50)	Not null	This field is a reference to table tbl_institution.
renewaldate	bigint(20)	Not null	This field stores the renewal date of the plan subscribed by the institution.
subscriptiondate	varchar(50)	Not null	This field store the subscription date by the institution.

6. Table Name: **tbl_coordinators**

Description: This table is used to store the coordinator information.

Table 3.6 tbl_coordinators

Field Name	**Data Type**	**Constraint**	**Description**
coordinatorid	int	Primary Key	This field is used to uniquely identify coordinators, and the values are automatically generated.
name	varchar(50)	Not null	This field stores the name of the coordinator.
email	varchar(50)	Not null	This field stores the email of the coordinator.
contactno	varchar(15)	Not null	This field stores the contact number of coordinator.
institutionid	int	Foreign key	This field is a reference to table tbl_institution.
image	Varchar(100)	Null	This field is to store the image of coordinator.
loginid	int	Foreign key	This field is a reference to table tbl_login.

7. Table Name: **tbl_fests**

Description: This table used to store the fest information.

Table 3.7 tbl_fests

Field Name	**Data Type**	**Constraints**	**Description**
festid	int	Primary Key	This field is used to uniquely identify fests, and the values are automatically generated
fest_name	Varchar(50)	Not null	This field stores the name of the fest.
fest_image	Varchar(100)	Null	This field stores the image of fest.
brochere	Varchar(400)	Null	This field stores the name of brochure.
startdate	date	Not null	This field stores the starting date of the fest.
enddate	date	Not null	This field stores the ending date of the fest.
fest_description	Varchar(500)	Null	This field stores the description about the fest
institution_id	int	Foreign key	This field is a reference to table tbl_institution.
category_id	int	Foreign key	This field is a reference to table tbl_category.
regdate	date	Not null	This field stores the registered date of the fest
status	Varchar(20)	Not null	This field stores the publish status of the fest

8. Table Name: tbl_assign

Description: This table is used to store coordinator assign information

Table 3.8 tbl_assign

Field Name	**Data Type**	**Constraint**	**Description**
assign_id	int	Primary Key	This field is used to uniquely identify parcel content, and the values are automatically generated.
competition_id	varchar(50)	Foreign key	This field is a reference to table tbl_competition.
coordinator_id	varchar(100)	Foreign key	This field is a reference to table tbl_coordinator.

9. Table Name: **tbl_guest_contact**

Description: This table used to store the fest information.

Table 3.9 tbl_guest_contact

Field Name	**Data Type**	**Constraints**	**Description**
contact_id	int	Primary Key	This field is used to uniquely identify messages, and the values are automatically generated
name	Varchar(50)	Not null	This field stores the name of the sender.
email	Varchar(50)	Not null	This field stores email of the sender.
phone_no	Varchar(10)	Not null	This field stores the phone number of the sender.
subject	Varchar(300)	Not null	This field stores the subject of the message.
message	Varchar(500)	Not null	This field stores the actual message the sender need to send.
status	Varchar(20)	Not null	This field stores read and unread status of the message for admin.

10. Table Name: tbl_competition

Description: This table is used to store competition information.

Table 3.10 tbl_competition

Field Name	**Data Type**	**Constraints**	**Description**
competition_id	int	Primary Key	This field is used to uniquely identify competitions and is automatically generated.
competition_name	Varchar(50)	Not null	This field stores the name of the competition.
image	Varchar(200)	Null	This field store the image of competition.
competition_date	date	Not null	This field stores the competition date.
description	Varchar(500)	Null	This field stores the description of competition.
type	varchar(50)	Not null	This field stores the type of competition.
max_members	int	Not null	This field stores the maximum number of participants allowed to a team competition.

min_members	int	Not null	This field stores the minimum number of participants required for the competition
grouplimit	int	Not null	This field stores the number of teams allowed to participate for a competition.
fest_id	int	Foreign key	This field is a reference to table tbl_fests.
reg_fee	int	Not null	This field stores the registration fee for the competition.
reg_date	date	Not null	This field stores the competition registration date.
res_status	Varchar(20)	Not null	This field stores completion status of the competition.
brochure	Varchar(300)	Null	This field stores brochure of the competition

11. Table Name: tbl_participant

Description: This table is used to store participant information.

Table 3.11 tbl_participant

Field Name	**Data Type**	**Constraint**	**Description**
participantid	int	Primary Key	This field is used to uniquely identify participant, and the values are automatically generated
participantname	varchar (100)	Not null	This field stores the name of the participant.
participantemail	Varchar(200)	Not null	This field stores the email of the participant.
contact_no	Varchar(15)	Not null	This field stores the contact number of participant.
participantimage	Varchar(200)	Null	This field stores the name of the participant image.
loginid	int	Foreign key	This field is a reference to table tbl_login.
academic_status	Varchar(20)	Not null	This field is used to store the academic status of the participant.
institution_name	Varchar(100)	Null	This field is used to store the institution name.
institution_id_proof	Varchar(100)	Null	This field is used to store the name of id proof.

12. Table Name: tbl_competitionreg

Description: This table is used to store competition registration information.

Table 3.12 tbl_competitionreg

Field Name	Data Type	Constraint	Description
competitionregid	int	Primary Key	This field is used to uniquely identify booking, and the values are automatically generated.
competitionid	int	Foreign Key	This field is a reference to the table tbl_competition.
chessno	int	Not null	This field stores the chess number of the participant.
attendance	Varchar(10)	Not null	This field stores the attendance status of the participant.
regdate	date	Not null	This field stores the date of registration.
participantid	int	Not null	This field is a reference to the table tbl_participant.

13. Table Name: tbl_competition_group_details

Description: This table is used to store group participant information.

Table 3.13 tbl_competition_group_details

Field Name	Data Type	Constraint	Description
detailid	int	Primary Key	This field is used to uniquely identify each participant of a group, and the values are automatically generated.
participantname	Varchar(50)	Not null	This field is used to store the name of the participant.
institutionidproof	Varchar(400)	Not null	This field is used to store the name of the id proof.
contact_no	Varchar(15)	Not null	This field is used to store the contact number.
email	Varchar(100)	Not null	This field is used to store the email.
competitionregid	int	Not null	This field is a reference to tbl_competitionreg

14. Table Name: tbl_result

Description: This table is used to store competition result information.

Table 3.14 tbl_result

Field Name	Data Type	Constraint	Description
result_id	int	Primary Key	This field is used to uniquely identify payment, and the values are automatically generated.
competiton_id	int	Foreign Key	This field is a reference to table tbl_competition.
Competition_reg_id	int	Not null	This field is a reference to table tbl_competitionreg.
position	Varchar(30)	Not null	This field is used to store the position of the participant

15. Table Name: tbl_payment

Description: This table is used to store payment information.

Table 3.15 tbl_result

Field Name	Data Type	Constraint	Description
payment_id	int	Primary Key	This field is used to uniquely identify payment, and the values are automatically generated.
payment_date	date	Not null	This field is used to store the payment date.
type	Varchar(30)	Not null	This field is stores the type of payment.
typeid	Varchar(30)	Not null	This field is used to store the type id, it can be plan id or competition reg id.
amount	Varchar(30)	Not null	This field is used to store the amount of payment

3.4 DATA FLOW DIAGRAM

3.4.1 Introduction to Data Flow Diagram

Data Flow Diagram is a network that describes the flow of data and processes that change, or transform, data throughout the system. This network is constructed by use a set of symbols that do not imply a physical implementation. It is a graphical tool for structured analysis of the system requirements. DFD models a system by using external entities from which data flows to a process, which transforms the data and creates, output-data-flows which go to other processes or external entities or files. Data in files may also flow to processes as inputs.

There are various symbols used in a DFD. Bubbles represent the processes. Named arrows indicate the data flow. External entities are represented by rectangles. Entities supplying data are known as sources and those that consume data are called sinks. Data are stored in a data store by a process in the system. Each component in a DFD is labelled with a descriptive name. Process names are further identified with a number.

The Data Flow Diagram shows the logical flow of a system and defines the boundaries of the system. For a candidate system, it describes the input (source), outputs (destination), database (files) and procedures (data flow), all in a format that meet the user's requirements.

The main merit of DFD is that it can provide an overview of system requirements, what data a system would process, what transformations of data are done, what files are used, and where the results flow.

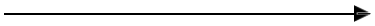
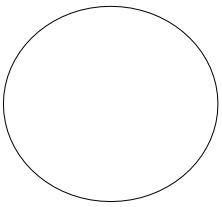
This network is constructed by use a set of symbols that do not imply a physical implementation. It is a graphical tool for structured analysis of the system requirements. DFD models a system by using external entities from which data flows to a process, which transforms the data and creates, output-data-flows which go to other processes or external entities or files. External entities are represented by rectangles. Entities supplying data are known as sources and those that consume data are called sinks. Data are stored in a data store by a process in the system. It is a graphical tool for structured analysis of the system requirements. DFD models a system by using external entities from which data flows to a

process, which transforms the data and creates, output-dataflows which go to other processes or external entities or files. Data in files may also flow to processes as inputs.

Rules for constructing a Data Flow Diagram

1. Arrows should not cross each other.
2. Squares, circles and files must bear names.
3. Decomposed data flow squares and circles can have same time
4. Choose meaningful names for data flow
5. Draw all data flows around the outside of the diagram

Basic Data Flow Diagram Symbols

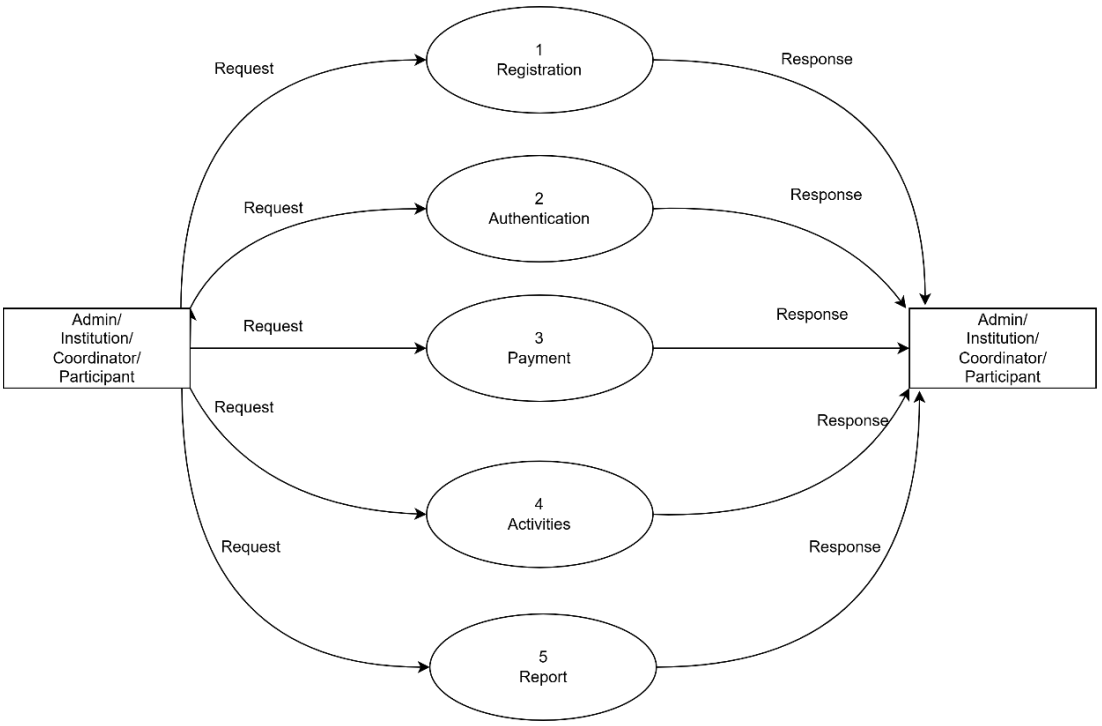
	A data flow is a route, which enables packets of data to travel from one point to another. Data may flow from a source to a process and from data store or process. An arrow line depicts the flow, with arrow head pointing in the direction of the flow
	Circles stands for process that converts data into information. A process represents transformation where incoming data flows are changed into outgoing data flows
	A data store is a repository of data that is to be stored for use by a one or more process may be as simple as buffer or queue or sophisticated as relational database. They should have clear names. If a process merely uses the content of store and does not alter it, the arrowhead goes only from the store to the process. If a process alters the details in the store, then a double-headed arrow is used
	A source or sink is a person or part of an organization, which enters or receives information from the system, but is considered to be outside the contest of data flow model.

3.4.2 Data Flow Diagram

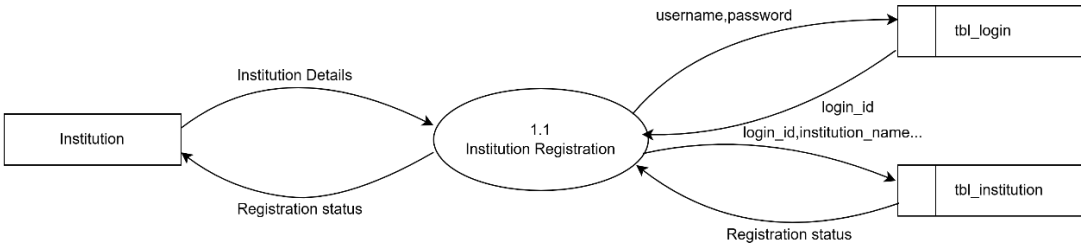
Each component in a DFD is labelled with a descriptive name. Process name are further identified with number. Context level DFD is draw first. Then the process is decomposed into several elementary levels and is represented in the order of importance. A DFD describes what data flow (logical) rather than how they are processed, so it does not depend on hardware, software, and data structure or file organization.

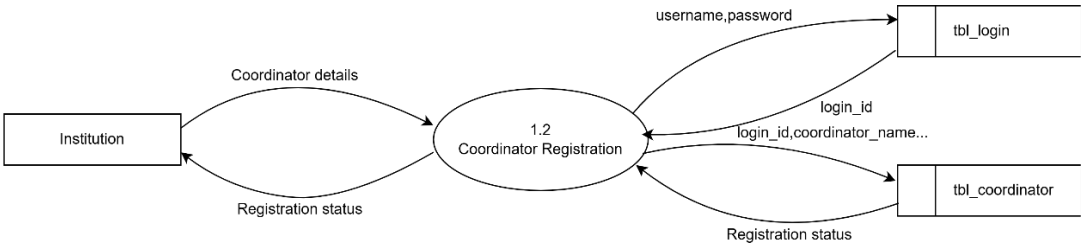
A DFD methodology is quite effective; especially when the required design.

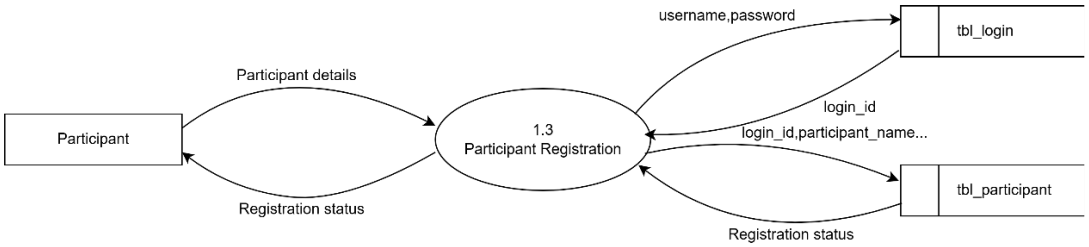
1.1 Zeroth Level DFD for Festivo

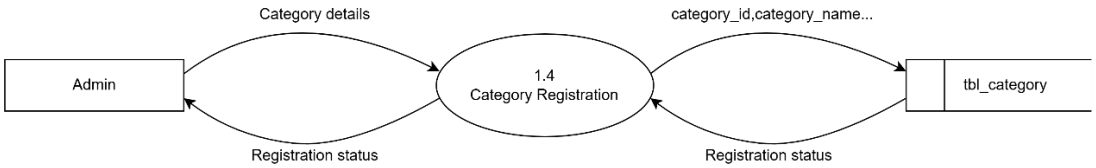


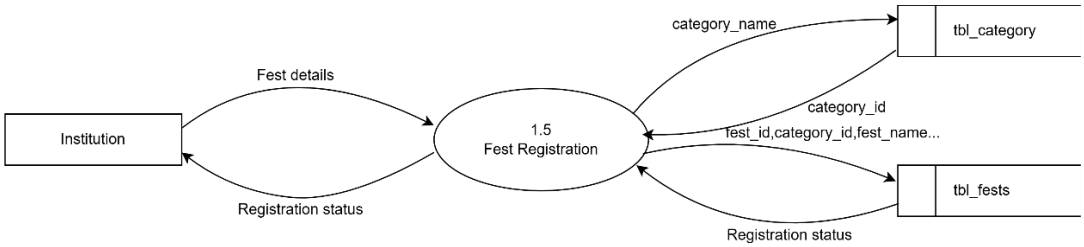
1.2 First Level DFD for Festivo

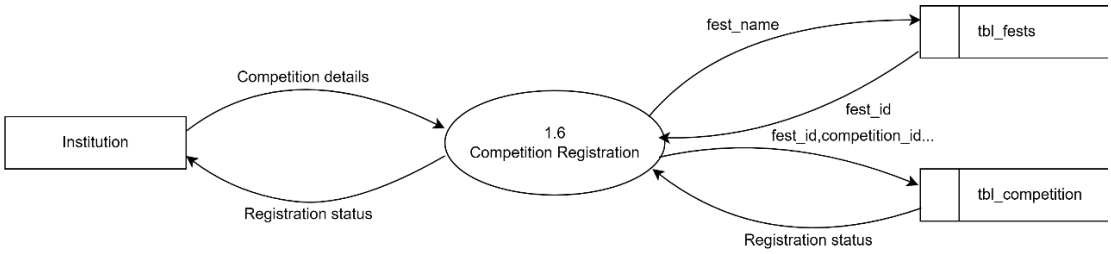

Institution
Institution Details
Registration status
1.1
Institution Registration
username,password
tbl_login
login_id
login_id,institution_name...
tbl_institution
Registration status


Institution
Coordinator details
Registration status
1.2
Coordinator Registration
username,password
tbl_login
login_id
login_id,coordinator_name...
tbl_coordinator
Registration status

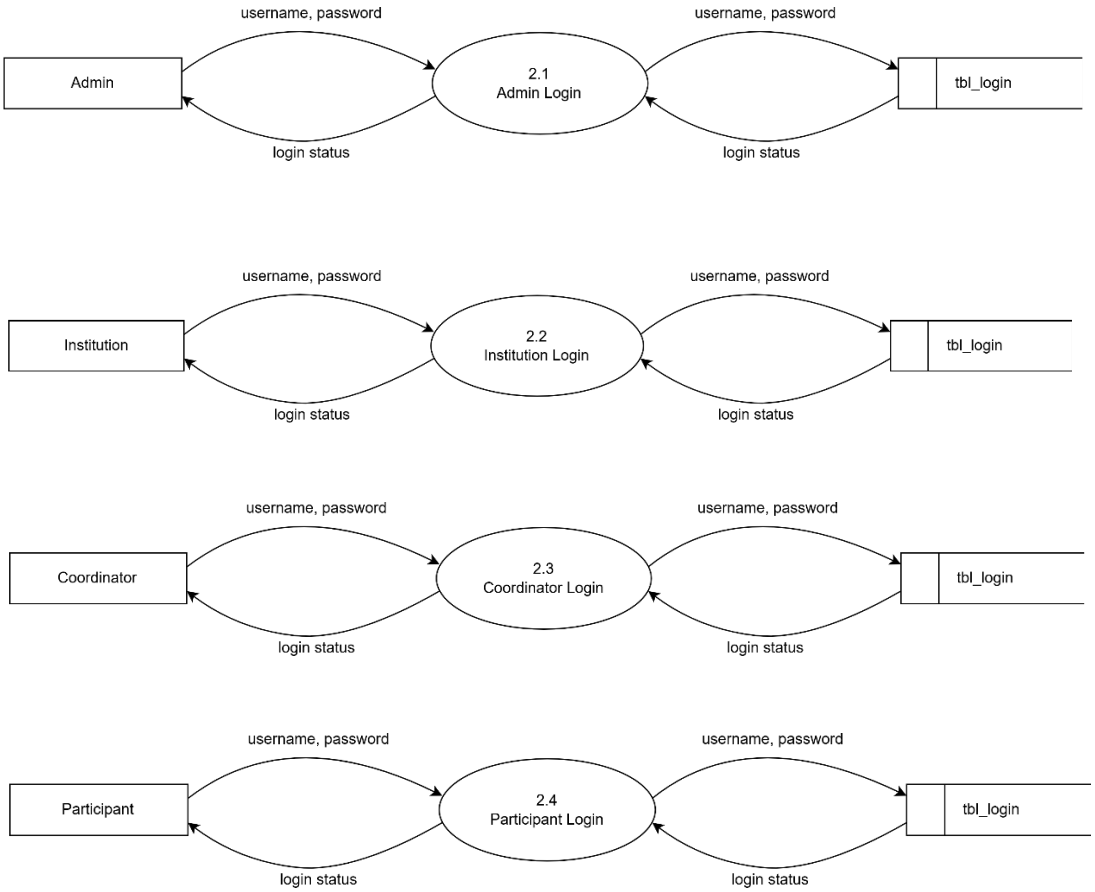

Participant
Participant details
Registration status
1.3
Participant Registration
username,password
tbl_login
login_id
login_id,participant_name...
tbl_participant
Registration status


Admin
Category details
Registration status
1.4
Category Registration
category_id,category_name...
tbl_category
Registration status

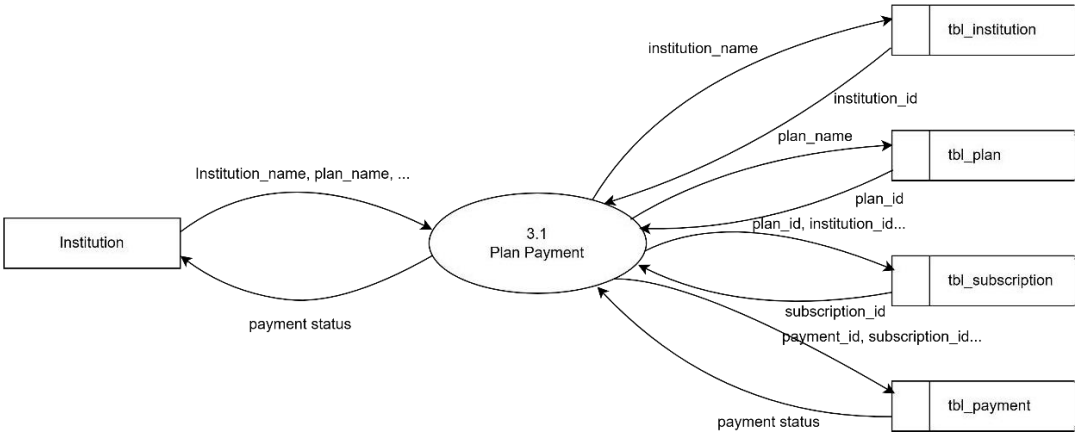

Institution
Fest details
Registration status
1.5
Fest Registration
category_name
tbl_category
category_id
fest_id,category_id,fest_name...
tbl_fests
Registration status

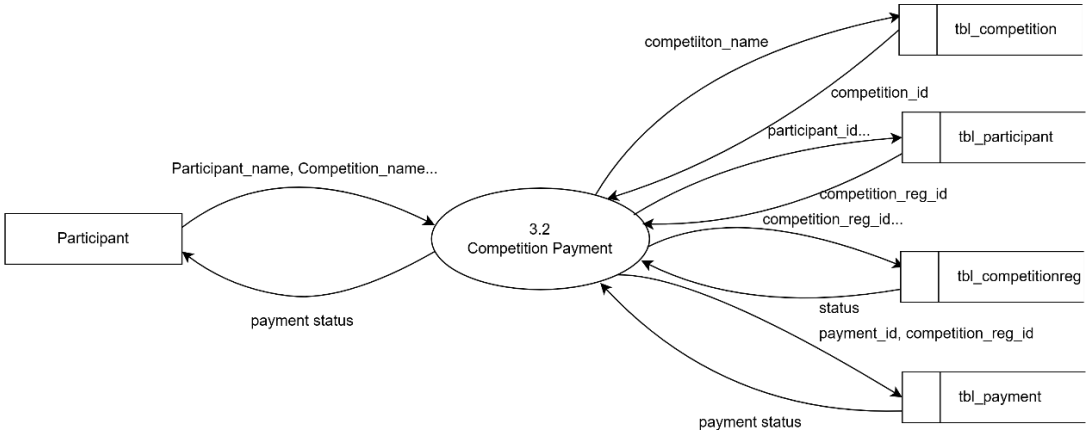


1.3 Second Level DFD for Registration

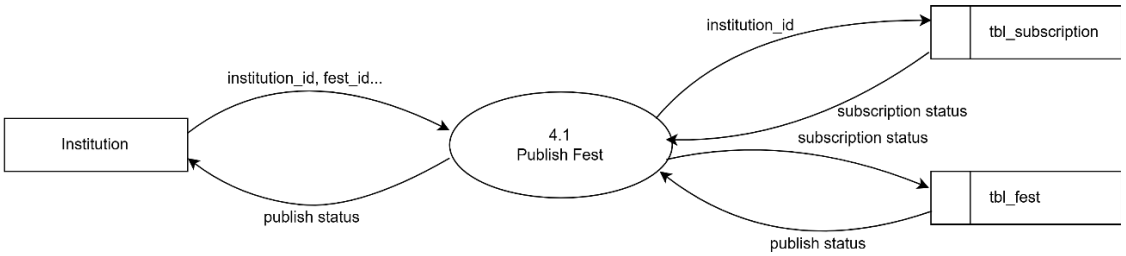


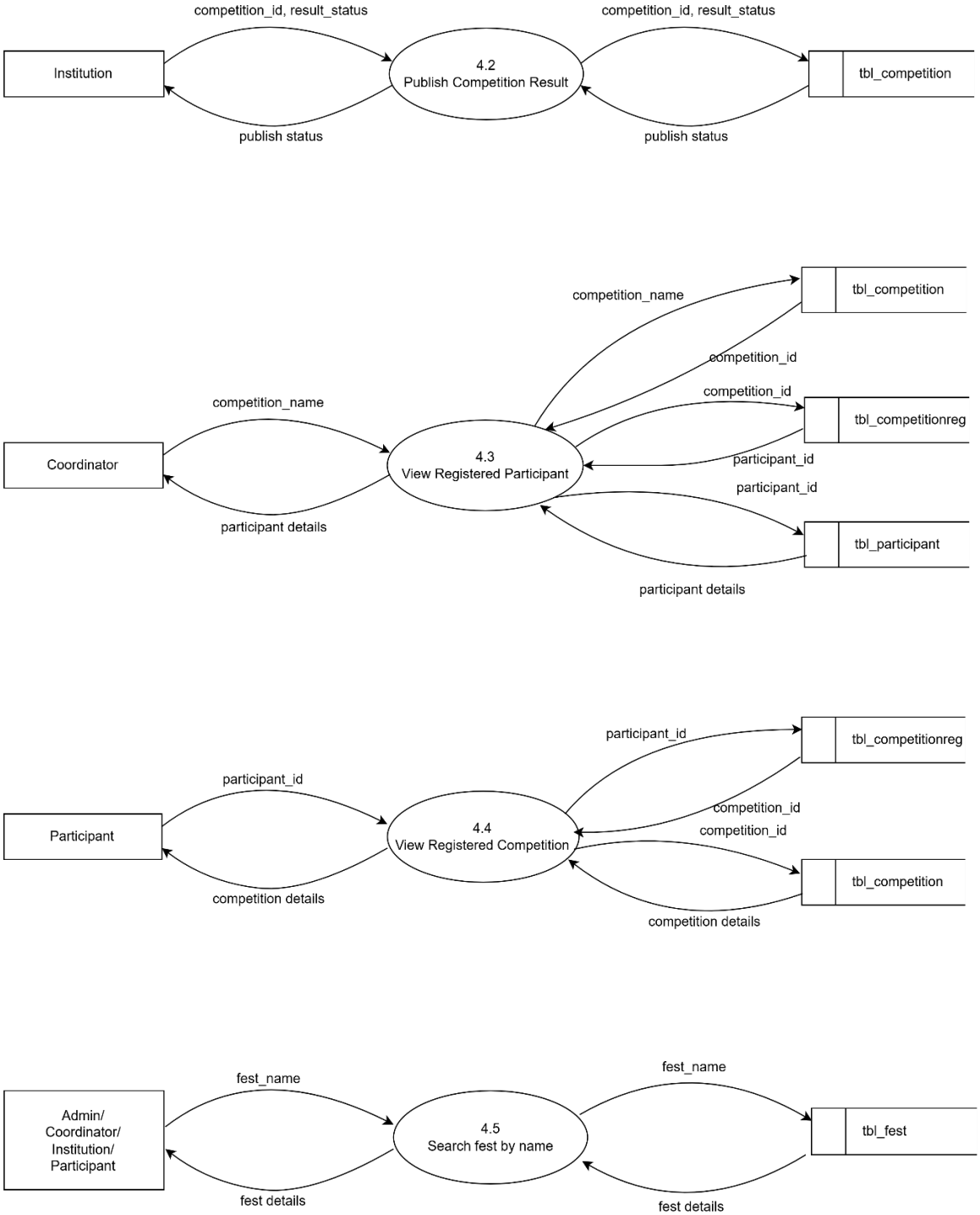
1.4 Second Level DFD for Authentication



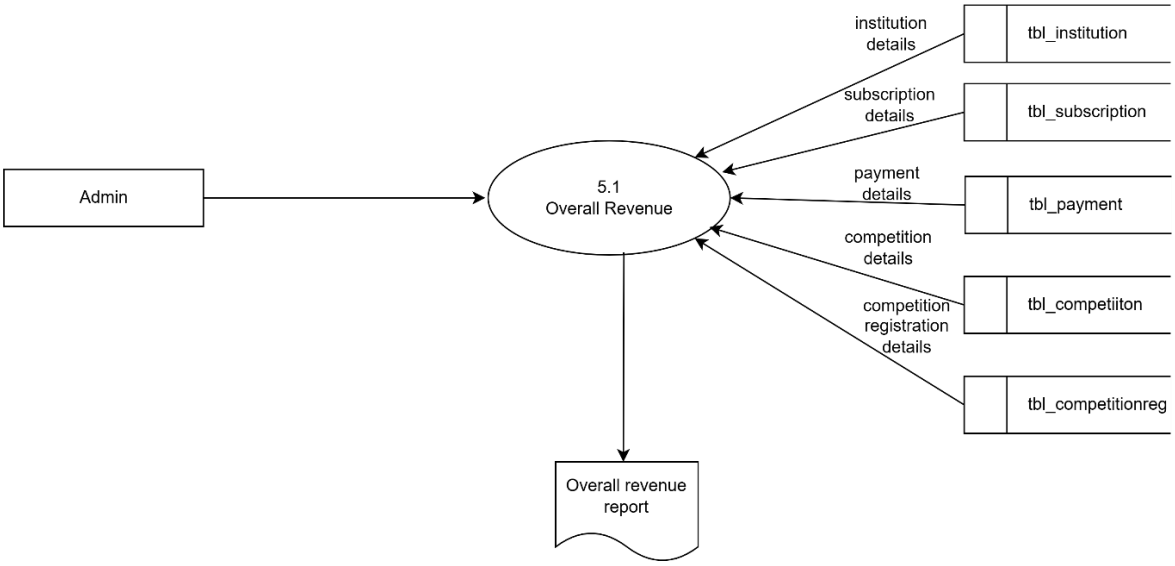


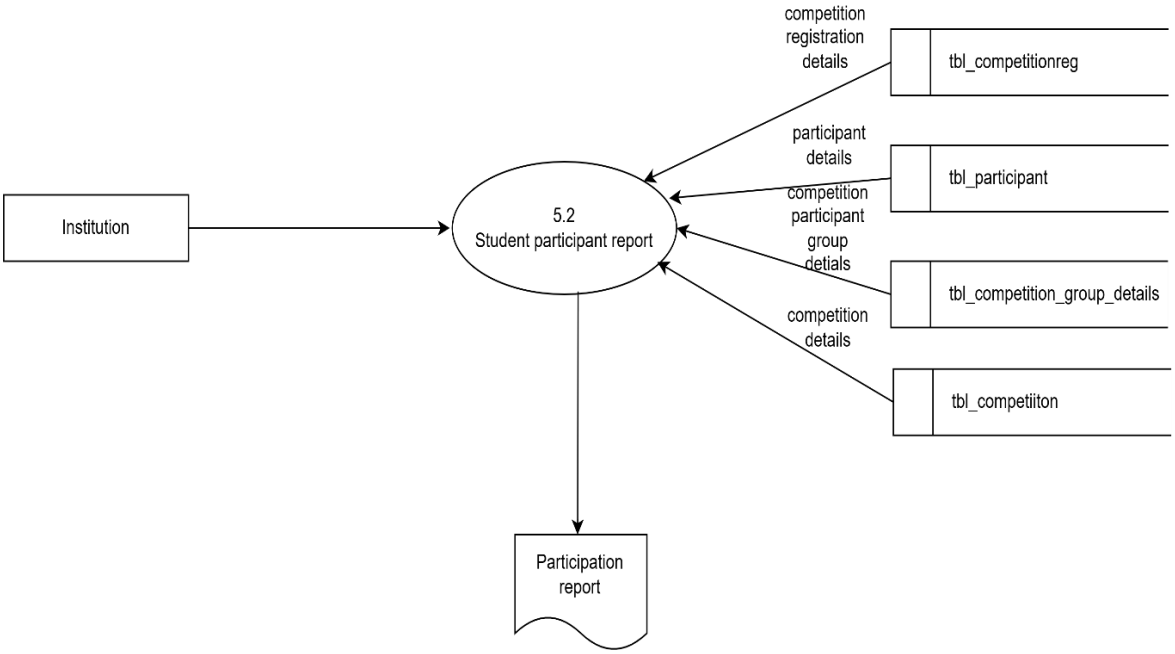
1.5 Second Level DFD for Payment

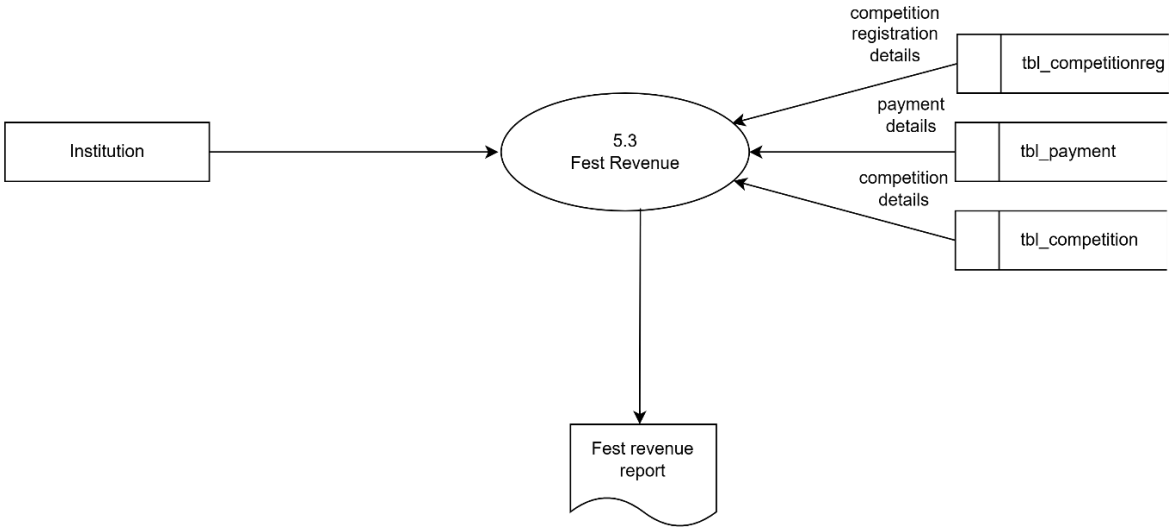


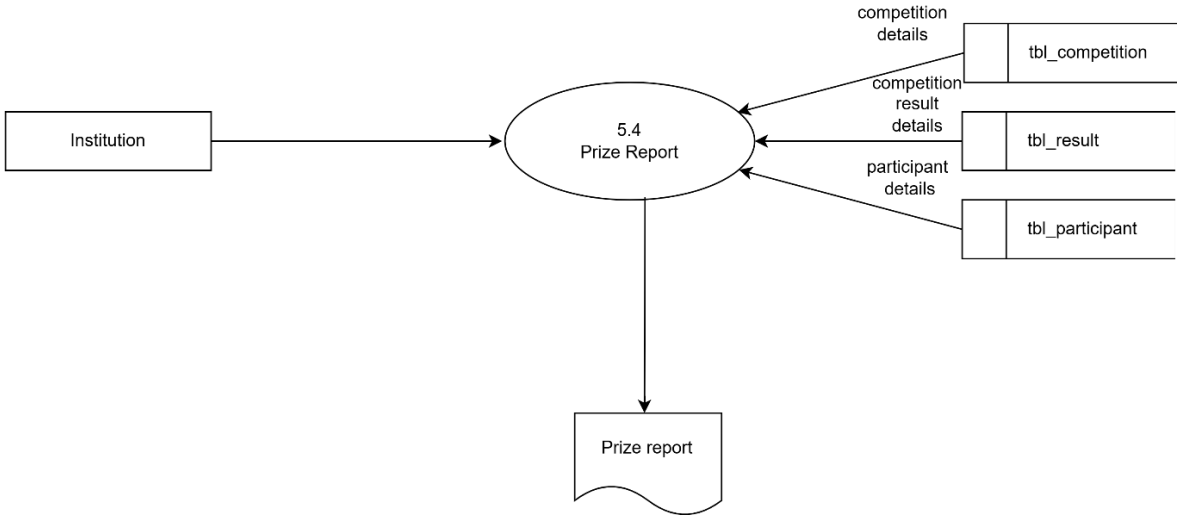


1.6 Second Level DFD for Activites


Admin
5.1
Overall Revenue
institution details
subscription details
payment details
competition details
competition registration details
tbl_institution
tbl_subscription
tbl_payment
tbl_competiiton
tbl_competitionreg
Overall revenue report


Institution
5.2
Student participant report
competition registration details
participant details
competition participant group detials
competition details
tbl_competitionreg
tbl_participant
tbl_competition_group_details
tbl_competiiton
Participation report





1.7 Second Level DFD for Reports

3.5 INTERFACE DESIGN

These modules can apply to hardware, software or the interface between a user and a machine. An example of a user interface could include a GUI, a control panel for a nuclear power plant, or even the cockpit of an aircraft. In systems engineering, all the inputs and outputs of a system, subsystem, and its components are listed in an interface control document often as part of the requirements of the engineering project. The development of a user interface is a unique field.

3.5.1 USER INTERFACE SCREEN DESIGN

The user interface design is very important for any application. The interface design describes how the software communicates within itself, to system that interpreted with it and with humans who use it. The input design is the process of converting the user oriented inputs into the computer based format. The data is fed into the system using simple inactive forms. The forms have been supplied with messages so that the user can enter data without facing any difficulty. They data is validated wherever it requires in the project. This ensures that only the correct data have been incorporated into system. The goal of designing input data is to make the automation as easy and free from errors as possible. For providing a good input design for the application easy data input and selection features are adopted. The input design requirements such as user friendliness, consistent format and interactive dialogue for giving the right messages and help for the user at right are also considered for development for this project. Input Design is a part of the overall design. The input methods can be broadly classified into batch and online. Internal controls must be established for monitoring the number of inputs and for ensuring that the data are valid. The basic steps involved in input design are:

- Review input requirements.
- Decide how the input data flow will be implemented.
- Decide the source document.
- Prototype on line input screens.
- Design the input screens.

The quality of the system input determines the quality of the system output. Input specifications describe the manner in which data enter the system for processing. Input design features can ensure the reliability of the system and produce results from accurate data. The input design also determines whether the user can interact efficiently with the system. The forms have been supplied with messages so that the user can enter data without facing any difficulty. They data is validated wherever it requires in the project. This ensures that only the correct data have been incorporated into system. These are sample input forms:

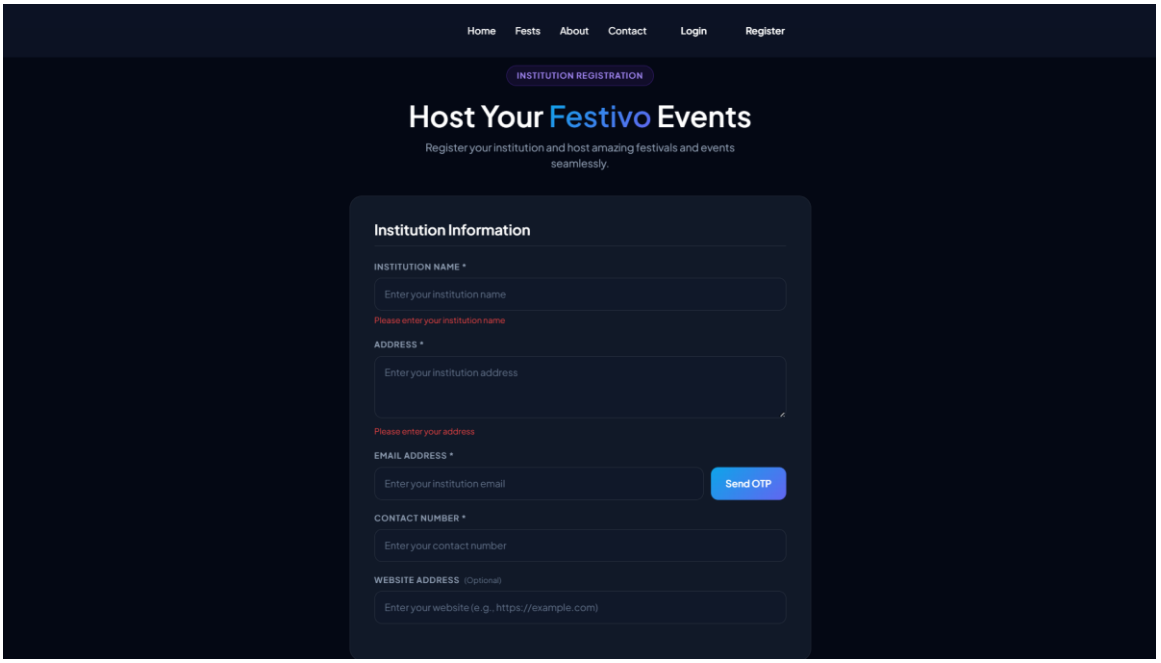


Fig. 3.8 Institution Registration Form

This input form is designed for registering a new institution in the Festivo platform. It contains textboxes for entering Institution Name, Contact Number, Email, Address, Username and Password. The form also provides a provision for the admin to select the institution type and location from the dropdown. After clicking the Submit button the institution is registered into the system. The institution registration form is very important in the project. Each field has its own label that denotes the value that needs to be entered in that box. Also, each textbox includes helpful placeholders for the user to understand the type of data to be entered in box. The form also has a button that allows the admin to save details entered in form to database table. The data entered in the form should be valid according to the type of each field.

3.5.2 OUTPUT DESIGN

A quality output is one, which meets the requirements of end user and presents the information clearly. In any system result of processing are communicated to the user and to the other system through outputs. In the output design it is determined how the information is to be displayed for immediate need. It is the most important and direct source information is to the user. Efficient and intelligent output design improves the system's relationships with the user and helps in decision -making. The objective of the output design is to convey the information of all the past activities, current status and to emphasis important events. The output generally refers to the results and information that is generated from the system. Outputs from computers are required primarily to communicate the results of processing to the users. Output also provides a means of storage by copying the results for later reference in consultation. There is a chance that some of the end users will not actually operate the input data or information through workstations, but will see the output from the system. Two phases of the output design are: 1. Output Definition 2. Output Specification Output Definition takes into account the type of output contents, its frequency and its volume, the appropriate output media is determined for output. Once the media is chosen, the detail specification of output documents are carried out. The nature of output required from the proposed system is determined during logical design stage. It takes the outline of the output from the logical design and produces output as specifiedduring the logical design phase.

In a project, when designing the output, the system analyst must accomplish the following:

- Determine the information to present.
- Decide whether to display, print, speak the information and select the output medium.
- Arrange the information in acceptable format.
- Decide how to distribute the output to the intended receipt.
- Thus, by following the above specifications, a high-quality output can be generated.

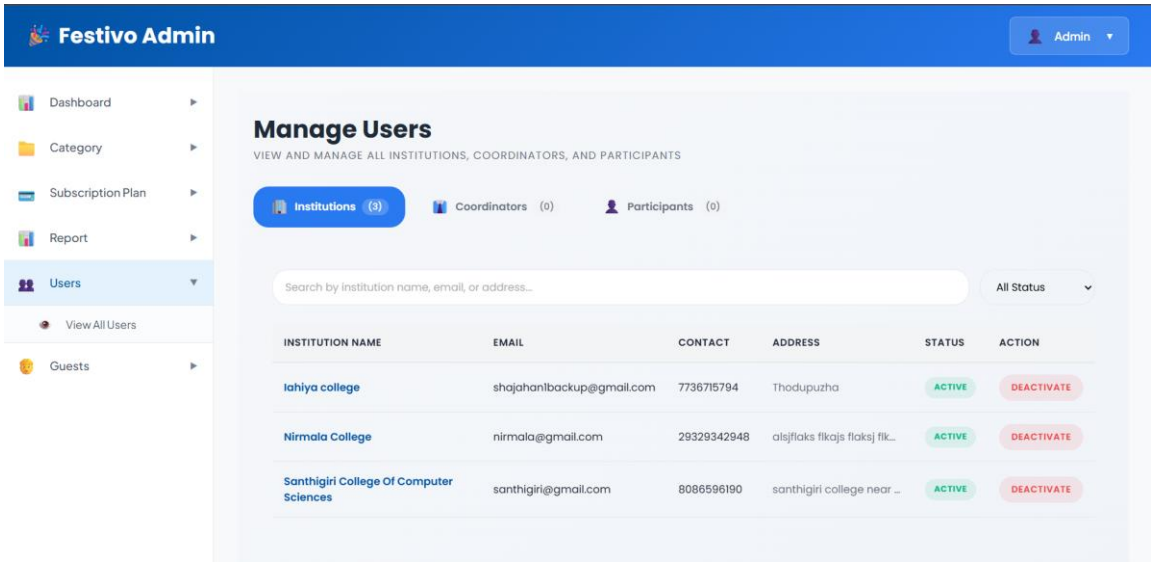


Fig. 3.9 View all users Page

This is the Admin view user page of the ‘Festivo’ platform that displays all registered users. Currently, the Institution filter is applied, so only institutions are shown. The page includes summary cards showing the total number of users by role and a search bar to quickly find users.

The main section contains a table with details such as institution name, username, contact number, role, status, and an action option to deactivate users. A navigation sidebar is provided to access other modules like Dashboard, Category, Subscription Plans, Reports, Users, and Guest. This page plays an important role in managing and monitoring user registrations in the system.

4. IMPLEMENTATION

Implementation is the stage of the project when the theoretical design is turned into a working system. The implementation stage is a systems project in its own rig ht. It includes careful planning, investigation of current system and its constraints on implementation, design of methods to achieve the changeover, training of the staff in the changeover procedure and evaluation of changeover method

4.1 CODING STANDARDS

Writing an efficient software code requires a thorough knowledge of programming. This knowledge can be implemented by following a coding style which comprises several guidelines that help in writing the software code efficiently and with minimum errors. These guidelines, known as coding guidelines, are used to implement individual programming language constructs, comments, formatting, and so on. These guidelines, if followed, help in preventing errors, controlling the complexity of the program, and increasing the readability and understandability of the program. A set of comprehensive coding guidelines encompasses all aspects of code development. To ensure that all developers work in a harmonized manner (the source code should reflect a harmonized style as a single developer had written the entire code in one session), the developers should be aware of the coding guidelines before starting a software project. Moreover, coding guidelines should state how to deal with the existing code when the software incorporates it or when maintenance is performed. Since there are numerous programming languages for writing software codes, each having different features and capabilities, coding style guidelines differ from one language to another. However, there are some basic guidelines which are followed in all programming languages. These include naming conventions, commenting conventions, and formatting conventions.

1. File header comments are useful in providing information related to a file as a whole and comprise identification information such as date of creation, Name of the creator, and a brief description of the software code

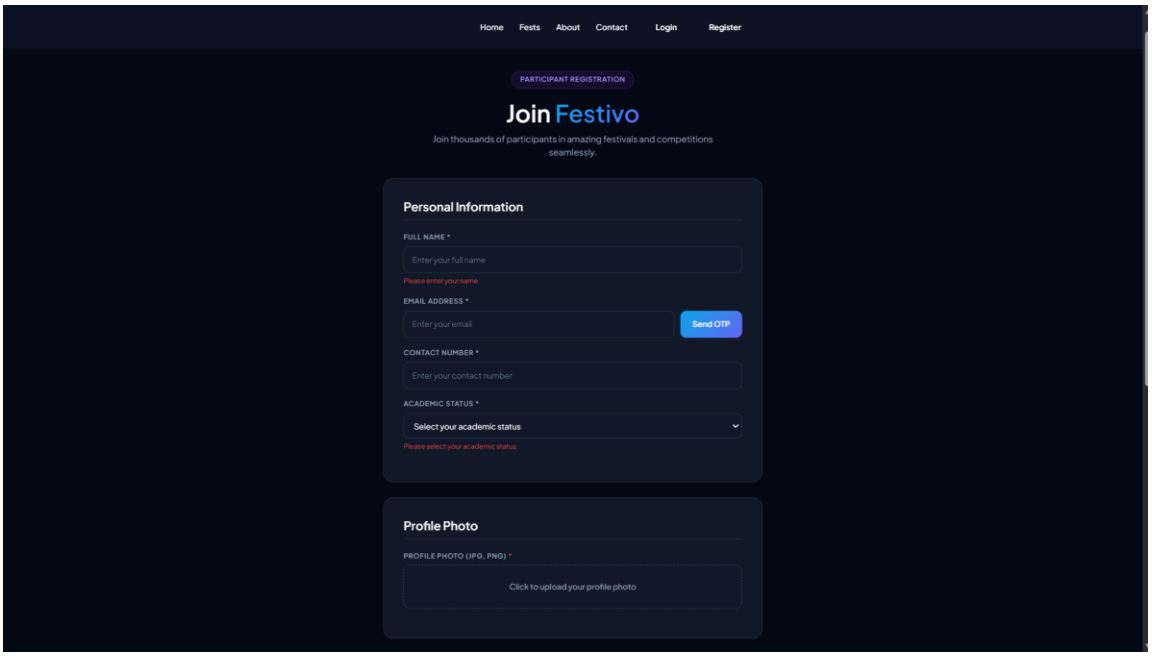
2. Trailing comments are used to provide explanation of a single line of code. These comments are used to clarify the complex code. These also specify the function of the

abbreviated variable names that are not clear. In some languages, trailing comments are used with the help of a double slash (//) .

3. Indentation: This refers to one or more spaces left at the beginning of statements in the program. Indentation is useful in making the code easily readable. However, the spaces used for indentation should be followed in the entire program.

4. Implementing coding guidelines: If coding guidelines are used in a proper manner, errors can be detected at the time of writing the software code. Such detection in early stages helps in increasing the performance of the software as well as reducing the additional and unplanned costs of correcting and removing errors. Moreover, if a well defined coding guideline is applied, the program yields a software system that is easy to comprehend and maintain.

4.2 SAMPLE CODE



4.1 Registration Form

```
//ts page
import { Component, ChangeDetectorRef, NgZone } from '@angular/core';
import { FormBuilder, FormGroup, Validators, ReactiveFormsModule } from
'@angular/forms';
import { CommonModule } from '@angular/common';
```

```
import { RouterLink, Router } from '@angular/router';
import { DbService } from '../../db-service';

@Component({
  selector: 'app-participant-register',
  standalone: true,
  imports: [ReactiveFormsModule, CommonModule, RouterLink],
  templateUrl: './participant-register.html',
  styleUrl: './participant-register.scss',
})
export class ParticipantRegister {
  registerForm: FormGroup;
  selectedImage: File | null = null;
  imagePreview: string | null = null;
  selectedIdProof: File | null = null;
  idProofPreview: string | null = null;
  isLoading: boolean = false;
  showSuccessModal: boolean = false;
  showErrorModal: boolean = false;
  modalMessage: string = '';
  modalTitle: string = '';

  constructor(
    private fb: FormBuilder,
    private dbService: DbService,
    private router: Router,
    private cdr: ChangeDetectorRef,
    private ngZone: NgZone
  ) {
```

```
    this.registerForm = this.fb.group({
      name: ['', Validators.required],
      username: ['', [Validators.required, Validators.minLength(3)]],
      email: ['', [Validators.required, Validators.email]],
      phone: ['', [Validators.required, Validators.pattern('^[0-9]{10,12}$')]],
      academicStatus: ['', Validators.required],
      institutionName: [''],
      institutionIdProof: [null],
      image: [null],
      password: ['', [Validators.required, Validators.minLength(6)]],
      confirmPassword: ['', Validators.required]
    }, { validators: this.passwordMatchValidator });


    // Listen to academic status changes to update institution_name and ID proof
validation
    this.registerForm.get('academicStatus')?.valueChanges.subscribe(value => {
      const institutionNameControl = this.registerForm.get('institutionName');
      const institutionIdProofControl = this.registerForm.get('institutionIdProof');
      if (value === 'school student' || value === 'college student') {
      institutionNameControl?.setValidators([Validators.required]);
      institutionIdProofControl?.setValidators([Validators.required]);
      } else {
        institutionNameControl?.clearValidators();
        institutionIdProofControl?.clearValidators();
      }
      institutionNameControl?.updateValueAndValidity();
      institutionIdProofControl?.updateValueAndValidity();
    });
  }
```

```
passwordMatchValidator(g: FormGroup) {
  return g.get('password')?.value === g.get('confirmPassword')?.value
    ? null : { 'mismatch': true };
}

onSubmit() {
  if (this.registerForm.valid) {
    this.isLoading = true;

    // Upload both files if selected
    Promise.all([
      this.selectedImage ? this.uploadImage() : Promise.resolve(''),
      this.selectedIdProof ? this.uploadIdProof() : Promise.resolve('')
    ]).then(([imageName, idProofName]) => {
      this.registerParticipant(imageName, idProofName);
    }).catch((error) => {
      console.error('Upload failed:', error);
      this.isLoading = false;
      this.showError('Upload Failed', 'Failed to upload files. Please try again.');
    });
  } else {
    this.markFormGroupTouched(this.registerForm);
  }
}

private uploadImage(): Promise<string> {
  return new Promise((resolve, reject) => {
  if (!this.selectedImage) {
      resolve('');
```

```
      return;
    }

    const formData = new FormData();
    formData.append('file', this.selectedImage);

    this.dbService.uploadImage(formData).subscribe({
      next: (response: any) => {
        if (response.message && response.filename) {
          resolve(response.filename);
        } else {
          reject('Upload failed');
        }
      },
      error: (error) => {
        reject(error);
      }
    });
  });
}

private uploadIdProof(): Promise<string> {
  return new Promise((resolve, reject) => {
  if (!this.selectedIdProof) {
      resolve('');
      return;
    }

    const formData = new FormData();
```

```
    formData.append('file', this.selectedIdProof);

    this.dbService.uploadImage(formData).subscribe({
      next: (response: any) => {
        if (response.message && response.filename) {
          resolve(response.filename);
        } else {
          reject('Upload failed');
        }
      },
      error: (error) => {
        reject(error);
      }
    });
  });
}

private registerParticipant(imageName: string, idProofName: string) {
  const formData = {
    participantname: this.registerForm.value.name,
    username: this.registerForm.value.username,
    participantemail: this.registerForm.value.email,
    contact_no: this.registerForm.value.phone,
    academic_status: this.registerForm.value.academicStatus,
    institution_name: this.registerForm.value.institutionName || '',
    institution_id_proof: idProofName,
    participantimage: imageName,
    password: this.registerForm.value.password
  };
```

```
console.log('Sending participant registration data:', formData);

this.dbService.registerParticipant(formData).subscribe({
  next: (response: any) => {
    console.log('Registration response:', response);
    this.isLoading = false;

    if (response.message === 'success') {
      console.log('Showing success modal');
      this.showSuccess('Registration Successful', 'Participant registered
successfully! You will be redirected to login.');
    } else if (response.message === 'username_exists') {
      console.log('Showing username exists error modal');
      this.showError('Username Taken', 'Username already exists. Please choose a
different username.');
    } else {
      console.log('Showing general error modal');
      this.showError('Registration Failed', 'Registration failed. Please try again.');
    }

    console.log('Modal states - Success:', this.showSuccessModal, 'Error:',
this.showErrorModal);
  },
  error: (error) => {
    this.isLoading = false;
    console.error('Registration error:', error);
    this.showError('Error', 'An error occurred during registration. Please try
again.');
  }
});
```

```
  }

  private markFormGroupTouched(formGroup: FormGroup) {
    Object.values(formGroup.controls).forEach(control => {
    control.markAsTouched();
      if ((control as any).controls) {
        this.markFormGroupTouched(control as FormGroup);
      }
    });
  }

  onImageSelected(event: any) {
    const file = event.target.files[0];
    if (file) {
      this.selectedImage = file;
      this.registerForm.patchValue({ image: file });

      // Create image preview
      const reader = new FileReader();
      reader.onload = () => {
        this.imagePreview = reader.result as string;
        this.cdr.detectChanges();
      };
      reader.readAsDataURL(file);
    }
  }

  removeImage() {
    this.selectedImage = null;
```

```
    this.imagePreview = null;
    this.registerForm.patchValue({ image: null });
  }

  onIdProofSelected(event: any) {
    const file = event.target.files[0];
    if (file) {
      this.selectedIdProof = file;
      this.registerForm.patchValue({ institutionIdProof: file });

      // Create preview
      const reader = new FileReader();
      reader.onload = () => {
        this.idProofPreview = reader.result as string;
        this.cdr.detectChanges();
      };
      reader.readAsDataURL(file);
    }
  }

  removeIdProof() {
    this.selectedIdProof = null;
    this.idProofPreview = null;
    this.registerForm.patchValue({ institutionIdProof: null });
  }

  showSuccess(title: string, message: string) {
    this.ngZone.run(() => {
      this.modalTitle = title;
```

```
    this.modalMessage = message;
    this.showSuccessModal = true;
    this.showErrorModal = false;
    console.log('showSuccess - Modal state updated:', {
    showSuccessModal: this.showSuccessModal,
    showErrorModal: this.showErrorModal,
      title: this.modalTitle,
      message: this.modalMessage
    });
    this.cdr.detectChanges();
  });
}

showError(title: string, message: string) {
  this.ngZone.run(() => {
    this.modalTitle = title;
    this.modalMessage = message;
    this.showErrorModal = true;
    this.showSuccessModal = false;
    console.log('showError - Modal state updated:', {
      showSuccessModal: this.showSuccessModal,
      showErrorModal: this.showErrorModal,
      title: this.modalTitle,
      message: this.modalMessage
    });
    this.cdr.detectChanges();
  });
}
```

```
  closeSuccessModal() {
    this.showSuccessModal = false;
    this.router.navigate(['/login']);
  }

  closeErrorModal() {
    this.showErrorModal = false;
  }
}

//html page
<div class="register-container">
   <div class="content-wrapper">
      <!-- Left Panel: Hero Content -->
      <div class="left-panel">
         <div class="hero-content">
            <h1>Join as a <br><span>Participant</span></h1>
            <p>Register now to participate in exciting competitions and events.
Showcase your talents, compete with
               the best, and create unforgettable memories.</p>
            <div class="hero-features">
               <div class="feature-item">
                  <i class="fas fa-check-circle"></i>
                  <span>Join Multiple Events</span>
               </div>
               <div class="feature-item">
                  <i class="fas fa-check-circle"></i>
                  <span>Track Your Progress</span>
               </div>
               <div class="feature-item">
                  <i class="fas fa-check-circle"></i>
```

```
                <span>Instant Registration</span>
            </div>
        </div>
    </div>
</div>

<!-- Right Panel: Registration Form -->
<div class="right-panel">
    <div class="form-card">
        <div class="form-header">
            <h2>Create Your Account</h2>
            <p>Enter your details below to get started</p>
        </div>

        <form [formGroup]="registerForm" (ngSubmit)="onSubmit()">
            <div class="form-grid">
                <!-- Participant Name -->
                <div class="form-group full-width">
                    <label for="name">Full Name</label>
                    <div class="input-wrapper">
                        <i class="fas fa-user"></i>
                        <input type="text" id="name" formControlName="name"
                            placeholder="e.g. John Doe">
                    </div>
                    <div class="error-msg"
                        *ngIf="registerForm.get('name')?.touched &&
registerForm.get('name')?.invalid">
                        Full Name is required
                    </div>
                </div>

                <!-- Email -->
```

```
                <div class="form-group">
                    <label for="email">Email Address</label>
                    <div class="input-wrapper">
                        <i class="fas fa-envelope"></i>
                        <input type="email" id="email" formControlName="email"
                            placeholder="john@example.com">
                    </div>
                    <div class="error-msg"
                        *ngIf="registerForm.get('email')?.touched &&
registerForm.get('email')?.invalid">
                        Please enter a valid email
                    </div>
                </div>

                <!-- Username -->
                <div class="form-group">
                    <label for="username">Username</label>
                    <div class="input-wrapper">
                        <i class="fas fa-user-circle"></i>
                        <input type="text" id="username"
formControlName="username"
                            placeholder="username">
                    </div>
                    <div class="error-msg"
                        *ngIf="registerForm.get('username')?.touched &&
registerForm.get('username')?.invalid">
                        Username is required (min 3 characters)
                    </div>
                </div>

                <!-- Phone -->
                <div class="form-group full-width">
```

```
                <label for="phone">Contact Number</label>
                <div class="input-wrapper">
                    <i class="fas fa-phone"></i>
                    <input type="tel" id="phone" formControlName="phone"
placeholder="+91 98765 43210">
                </div>
                <div class="error-msg"
                    *ngIf="registerForm.get('phone')?.touched &&
registerForm.get('phone')?.invalid">
                    Valid contact number is required (10-12 digits)
                </div>
            </div>

            <!-- Academic Status -->
            <div class="form-group full-width">
                <label for="academicStatus">Academic Status</label>
                <div class="input-wrapper">
                    <i class="fas fa-graduation-cap"></i>
                    <select id="academicStatus"
formControlName="academicStatus">
                        <option value="" disabled selected>Select your
status</option>
                        <option value="school student">School Student</option>
                        <option value="college student">College Student</option>
                        <option value="all">Other (Not a student)</option>
                    </select>
                </div>
                <div class="error-msg"
                    *ngIf="registerForm.get('academicStatus')?.touched  &&
registerForm.get('academicStatus')?.invalid">
                    Please select your academic status
                </div>
```

```
                </div>

                <!-- School/College Name (Conditional) -->
                <div class="form-group full-width"
*ngIf="registerForm.get('academicStatus')?.value === 'school student' ||
registerForm.get('academicStatus')?.value === 'college student'">
                    <label for="institutionName">
                        {{ registerForm.get('academicStatus')?.value === 'school
student' ? 'School Name' : 'College Name' }}
                    </label>
                    <div class="input-wrapper">
                        <i class="fas fa-school"></i>
                        <input type="text" id="institutionName"
formControlName="institutionName"
                            [placeholder]="registerForm.get('academicStatus')?.value
=== 'school student' ? 'e.g. St. Mary\'s High School' : 'e.g. ABC College'">
                    </div>
                    <div class="error-msg"
                        *ngIf="registerForm.get('institutionName')?.touched &&
registerForm.get('institutionName')?.invalid">
                        {{ registerForm.get('academicStatus')?.value === 'school
student' ? 'School name is required' : 'College name is required' }}
                    </div>
                </div>

                <!-- Institution ID Proof (Conditional) -->
                <div class="form-group full-width"
*ngIf="registerForm.get('academicStatus')?.value === 'school student' ||
registerForm.get('academicStatus')?.value === 'college student'">
                    <label for="idProof">Institution ID Proof <span style="color:
#e74c3c;">*</span></label>
                    <div class="image-upload-wrapper">
```

```
                    <input type="file" id="idProof"
accept="image/*,application/pdf" (change)="onIdProofSelected($event)"
                        style="display: none;" #idProofInput>
                    <div class="upload-area" *ngIf="!idProofPreview"
(click)="idProofInput.click()">
                        <i class="fas fa-id-card"></i>
                        <p>Click to upload ID proof</p>
                        <small>ID Card, Bonafide, etc. (PNG, JPG, PDF - Max
5MB)</small>
                    </div>
                    <div class="image-preview" *ngIf="idProofPreview">
                        <img [src]="idProofPreview" alt="ID Proof"
 *ngIf="!selectedIdProof?.type?.includes('pdf')">
                         <div class="pdf-preview"
 *ngIf="selectedIdProof?.type?.includes('pdf')">
                            <i class="fas fa-file-pdf"></i>
                            <p>{{ selectedIdProof?.name }}</p>
                        </div>
                        <button type="button" class="remove-image"
(click)="removeIdProof()">
                            <i class="fas fa-times"></i>
                        </button>
                    </div>
                </div>
                <div class="error-msg"
                    *ngIf="registerForm.get('institutionIdProof')?.touched &&
registerForm.get('institutionIdProof')?.invalid">
                    Institution ID proof is required
                </div>
            </div>

            <!-- Participant Image -->
```

```
                <div class="form-group full-width">
                    <label for="image">Profile Photo <span style="opacity:
0.6;">(Optional)</span></label>
                    <div class="image-upload-wrapper">
                        <input type="file" id="image" accept="image/*"
(change)="onImageSelected($event)"
                            style="display: none;" #fileInput>
                        <div class="upload-area" *ngIf="!imagePreview"
(click)="fileInput.click()">
                            <i class="fas fa-cloud-upload-alt"></i>
                            <p>Click to upload photo</p>
                            <small>PNG, JPG, JPEG (Max 5MB)</small>
                        </div>
                        <div class="image-preview" *ngIf="imagePreview">
                            <img [src]="imagePreview" alt="Profile Photo">
                            <button type="button" class="remove-image"
(click)="removeImage()">
                                <i class="fas fa-times"></i>
                            </button>
                        </div>
                    </div>
                </div>

                <!-- Password -->
                <div class="form-group">
                    <label for="password">Password</label>
                    <div class="input-wrapper">
                        <i class="fas fa-lock"></i>
                        <input type="password" id="password"
formControlName="password" placeholder="••••••••">
                    </div>
                    <div class="error-msg"
```

```
                    *ngIf="registerForm.get('password')?.touched &&
registerForm.get('password')?.invalid">
                    Password must be at least 6 characters
                  </div>
                </div>

                <!-- Confirm Password -->
                <div class="form-group">
                  <label for="confirmPassword">Confirm Password</label>
                  <div class="input-wrapper">
                    <i class="fas fa-lock"></i>
                    <input type="password" id="confirmPassword"
formControlName="confirmPassword"
                      placeholder="••••••••">
                  </div>
                  <div class="error-msg"
                    *ngIf="registerForm.get('confirmPassword')?.touched &&
registerForm.hasError('mismatch')">
                    Passwords do not match
                  </div>
                </div>
              </div>

              <button type="submit" class="submit-btn"
[disabled]="registerForm.invalid || isLoading">
                <span *ngIf="!isLoading">Register Now <i class="fas fa-arrow-
right"></i></span>
                <span *ngIf="isLoading"><i class="fas fa-spinner fa-spin"></i>
Registering...</span>
              </button>

              <div class="form-footer">
```

```
                    <p>Already have an account? <a routerLink="/login">Login
here</a></p>
                  </div>
                </form>
            </div>
        </div>
    </div>
</div>

<!-- Success Modal -->
<div class="modal-overlay" *ngIf="showSuccessModal"
(click)="closeSuccessModal()">
    <div class="modal-content success-modal" (click)="$event.stopPropagation()">
        <div class="modal-icon success">
            <i class="fas fa-check-circle"></i>
        </div>
        <h3>{{ modalTitle }}</h3>
        <p>{{ modalMessage }}</p>
        <button class="modal-btn primary" (click)="closeSuccessModal()">
            <i class="fas fa-arrow-right"></i> Go to Login
        </button>
    </div>
</div>

<!-- Error Modal -->
<div class="modal-overlay" *ngIf="showErrorModal"
(click)="closeErrorModal()">
    <div class="modal-content error-modal" (click)="$event.stopPropagation()">
        <div class="modal-icon error">
            <i class="fas fa-times-circle"></i>
        </div>
        <h3>{{ modalTitle }}</h3>
```

```
        <p>{{ modalMessage }}</p>
        <button class="modal-btn danger" (click)="closeErrorModal()">
            <i class="fas fa-redo"></i> Try Again
        </button>
    </div>
</div>



//js page
var express = require('express');
var router = express.Router();
var mysql = require('mysql');
var con = mysql.createConnection({
host: "localhost",
user: "root",
password: "",
database: "festivo",
});
/* GET users listing. */ router.post('/', function (req, res, next) {
    console.log('=== Participant Registration Request ===');
    console.log('Request body:', req.body);
    // First, check if username already exists
    var checkUsernameSql = "SELECT * FROM tbl_login WHERE username =
?";
    con.query(checkUsernameSql, [req.body.username], function (err, result) {
        if (err) {
            console.error('Check username error:', err);
            return res.send({ message: 'error', error: err.message });
        }
```

```
        if (result.length > 0) {
            console.log('Username already exists:', req.body.username);
            return res.send({ message: 'username_exists' });
        }
        console.log('Username available, proceeding with registration...');
        // Username doesn't exist, proceed with registration
        // Insert into tbl_login first
        var loginSql = "INSERT INTO tbl_login (username, password, role, status)
VALUES (?, ?, 'Participant', 'active')";
        con.query(loginSql, [req.body.username, req.body.password], function (err,
loginResult) {
            if (err) {
                console.error('Login creation error:', err);
                return res.send({ message: 'error', error: err.message });
            }
            var loginId = loginResult.insertId;
            console.log('Login created with ID:', loginId);
            // Insert into tbl_participant
            var participantSql = "INSERT INTO tbl_participant (participantname,
participantemail, contact_no, academic_status, institution_name,
institution_id_proof, participantimage, loginid) VALUES (?, ?, ?, ?, ?, ?, ?, ?)";
            var values = [
                req.body.participantname,
                req.body.participantemail,
                req.body.contact_no,
                req.body.academic_status,
                req.body.institution_name || '',
                req.body.institution_id_proof || '',
                req.body.participantimage || '',
                loginId
            ];
```

```
        console.log('Inserting participant with values:', values);
        con.query(participantSql, values, function (err, participantResult) {
            if (err) {
                console.error('Participant creation error:', err);
                console.error('Error details:', err);
                // Rollback: delete the login entry
                con.query("DELETE FROM tbl_login WHERE login_id = ?",
[loginId], function() {});
                return res.send({ message: 'error', error: err.message });
            }
            console.log('Participant registered successfully with ID:',
participantResult.insertId);
            res.send({
                message: 'success',
                participant_id: participantResult.insertId,
                login_id: loginId
            });
        });
    });
  });
});
module.exports=router;
```

5. TESTING

Coding conventions are a set of guidelines for a specific programming language that recommend programming style, practices and methods for each aspect of a piece program written in this language. These conventions usually cover file organization, indentation, comments, declarations, statements, white space, naming conventions, programming practices, programming principles, programming rules of thumb, architectural best practices, etc. These are guidelines for software structural quality. Software programmers are highly recommended to follow these guidelines to help improve the readability of their source code and make software maintenance easier.

5.1 TEST CASES

The objective of system testing is to ensure that all individual programs are working as expected, that the programs link together to meet the requirements specified and to ensure that the computer system and the associated clerical and other procedures work together. The initial phase of system testing is the responsibility of the analyst who determines what conditions are to be tested, generates test data, produced a schedule of expected results, runs the tests and compares the computer produced results with the expected results with the expected results. The analyst may also be involved in procedures testing. When the analyst is satisfied that the system is working properly, he hands it over to the users for testing. The importance of system testing by the user must be stressed. Ultimately it is the user must verify the system and give the go-ahead. During testing, the system is used experimentally to ensure that the software does not fail, i.e., that it will run according to its specifications and in the way users expect it to. Special test data is input for processing (test plan) and the results are examined to locate unexpected results. A limited number of users may also be allowed to use the system so analysts can see whether they try to use it in unexpected ways. It is preferably to find these surprises before the organization implements the system and depends on it. In many organizations, testing is performed by person other than those who write the original programs. Using persons who do not know how certain parts were designed or programmed ensures more complete and unbiased testing and more reliable software. Parallel running is often regarded as the final phase of system testing. Since he parallel operation of two systems is very demanding in terms of user resources it should be embarked on only if the user is satisfied with the results of

testing -- it should not be started if problems are known to exist. Testing is the major quality control measure during software development. Its basic function is to detect errors in the software. Thus the goal of testing is to uncover requirement design and coding errors in the program. Testing is the process of correcting a program with intends of finding an error. Different types of testing are,

1. Unit Testing
2. Integrated Testing
3. Black Box Testing
4. White Box Testing
5. Validation Testing
6. User Acceptance Testing

5.1.1 UNIT TESTING

In computer programming, unit testing is a method by which individual units of source code, sets of one or more computer program modules together with associated control data, usage procedures, and operating procedures are tested to determine if they are fit for use. In this testing we test each module individual and integrated the overall system. Unit testing focuses verification efforts on the smaller unit of software design in the module. This is also known as module testing. The modules of the system are tested separately. The testing is carried out during programming stage itself. In this testing step each module is found to working satisfactory as regard to the expected output from the module. There are some validation checks for verifying the data input given by the user which both the formal and validity of the entered. It is very easy to find error debug the system.

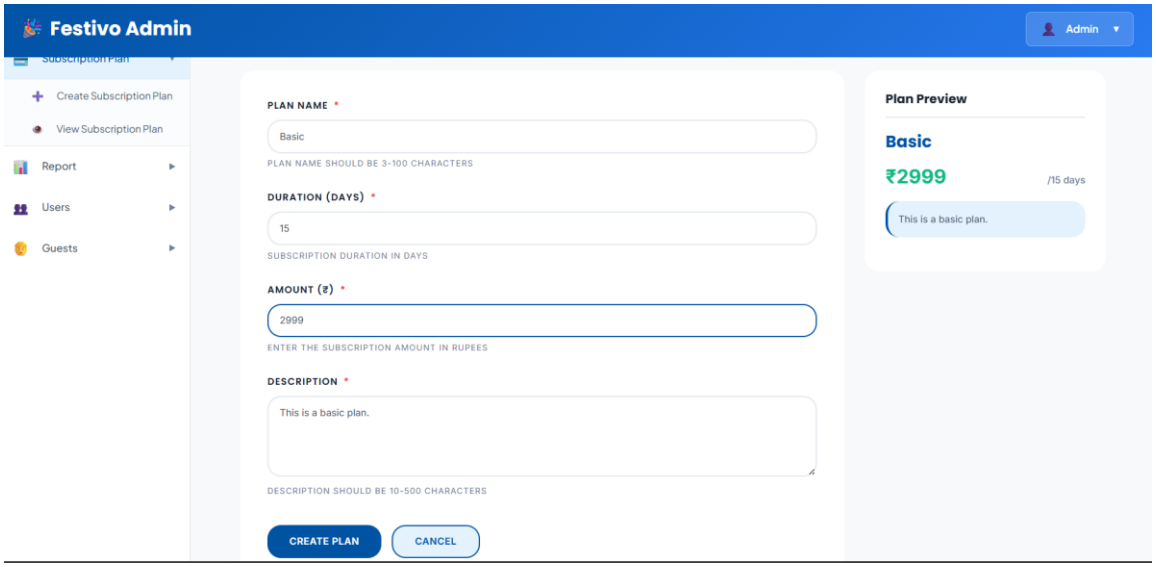


Fig. 5.1 Unit Testing

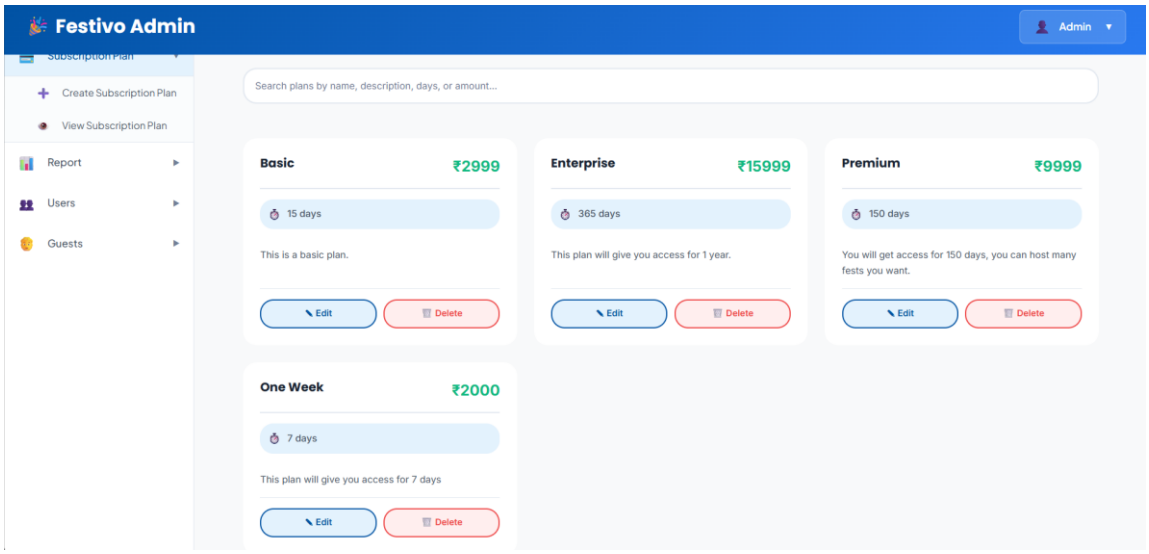


Fig. 5.2 Unit Testing Result

We have continued Unit Testing from the starting of the coding phase itself. Whenever we completed one small sub module, some amount of testing was done based on the requirements to see if the functionality is aligned to the gathered requirements.

5.1.2 INTEGRATION TESTING

Integration testing (sometimes called integration and testing, abbreviated I&T) is the phase in software testing in which individual software modules are combined and tested as a group. Software components may be integrated in an iterative way or all together ("big bang"). Normally the former is considered a better practice since it allows interface issues to be located more quickly and fixed. Data can be lost across an interface; one module can have an adverse effort on the other sub functions when combined by, may not produce the desired major functions. Integrated testing is the systematic testing for constructing the uncover errors within the interface. This testing was done with sample data. The developed system has run success full for this sample data. The need for integrated test is to find the overall system performance. Integration testing is a logical extension of unit testing. In its simplest form, two units that have already been tested are combined into a component and the interface between them is tested. A component, in this sense, refers to an integrated aggregate of more than one unit. Integration testing identifies problems that occur when units are combined. By using a test plan that requires you to test each unit and ensure the viability of each before combining units, you know that any errors discovered when combining units are likely related to the interface between units. This method reduces the number of possibilities to a far simpler level of analysis. Progressively larger groups of tested software components corresponding to elements of the architectural design are integrated and tested until the software works as a system.

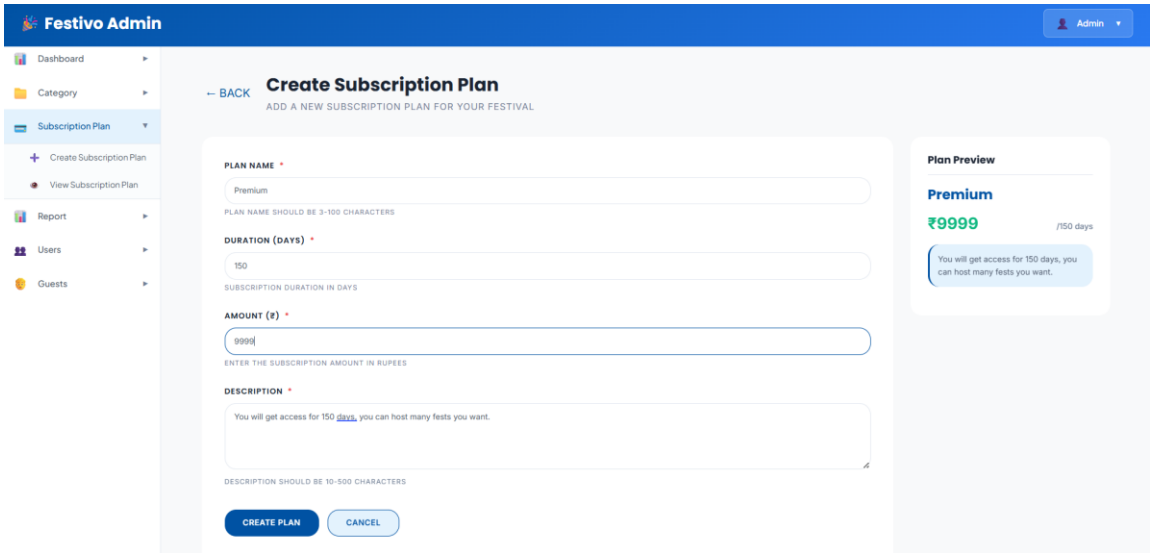


Fig. 5.3 Integration Testing

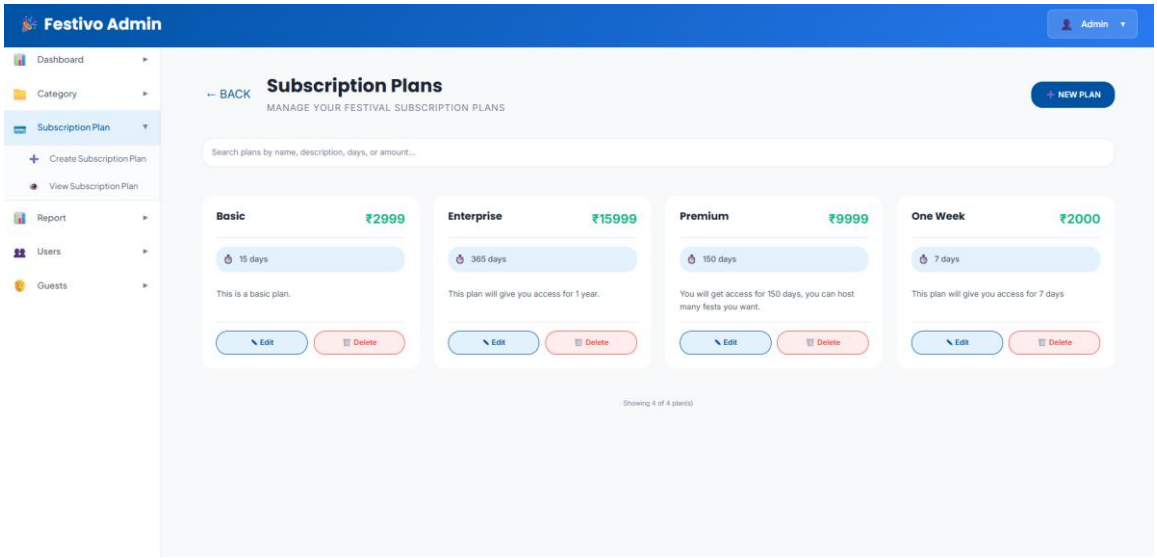


Fig. 5.4 Integration Testing Result

We have performed integration testing whenever we have combined two modules together. When two modules are combined, we have checked whether the functionality works correctly or not through integration testing.

5.1.3 VALIDATION TESTING

At the culmination of Black Box testing, software is completely assembled as a package, interface errors have been uncovered and corrected and final series of software tests, Validation tests begins. Validation testing can be defined many was but a simple definition is that validation succeeds when the software functions in a manner that can be reasonably accepted by the customer. After validation test has been conducted one of the two possible conditions exists. 1. The function or performance characteristics confirm to specification and are accepted. 2. A derivation from specification uncovered and a deficiency list is created.

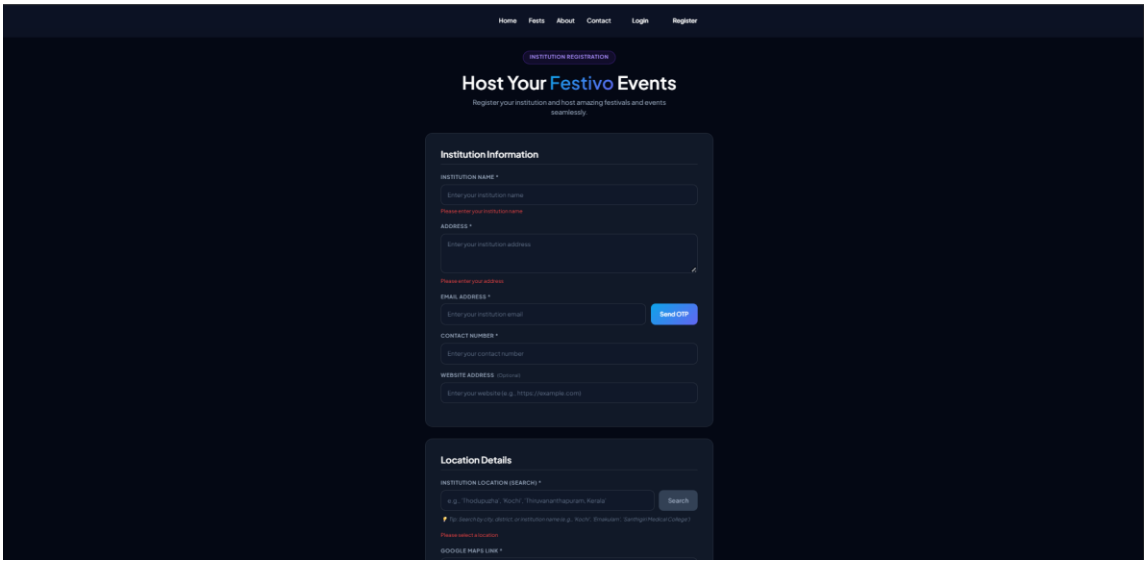


Fig. 5.5 Registration Field Validation

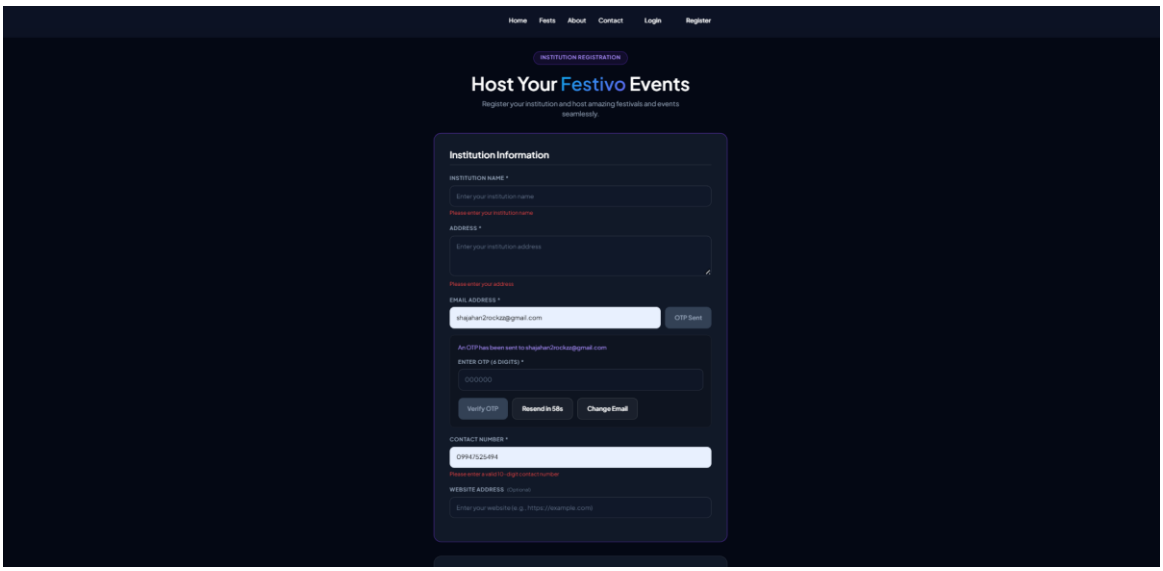


Fig 5.6 Registration Email Validation

5.1.4 USER ACCEPTANCE TESTING

Acceptance Testing is a level of the software testing process where a system is tested for acceptability. User Acceptance testing is the software testing process where system tested for acceptability & validates the end to end business flow. Such type of testing executed

by client in separate environment & confirms whether system meets the requirements as per requirement specification or not. UAT is performed after System Testing is done and all or most of the major defects have been fixed. This testing is to be conducted in the final stage of Software Development Life Cycle (SDLC) prior to system being delivered to a live environment. UAT users or end users are concentrating on end to end scenarios & typically involves running a suite of tests on the completed system. User Acceptance testing also known as Customer Acceptance testing (CAT), if the system is being built or developed by an external supplier. The CAT or UAT are the final confirmation from the client before the system is ready for production. The business customers are the primary owners of these UAT tests. These tests are created by business customers and articulated in business domain languages. So ideally it is collaboration between business customers, business analysts, testers and developers. It consists of test suites which involve multiple test cases & each test case contains input data (if required) as well as the expected output. The result of test case is either a pass or fail.

5.2 TEST CASE DOCUMENTS

A test case is a set of conditions or variables under which a tester will determine whether a system under test satisfies requirements or works correctly. The process of developing test cases can also help find problems in the requirements or design an application. A sample of test case document format is given below.

Table 5.1 Test Case

TC No.	Test Steps	Expected Result	Actual Result	Status	Comment
1.	Run application and navigate to Subscription plan registration screen.	Subscription plan registration screen is displayed. Fields for plan name, price, duration, description and a Submit button should be present.	Subscription registration screen is displayed with all required fields and Submit button.	Pass	

2.	Enter the submit button without entering the name field.	A message should be displaying stating that 'Plan name required' in beside of the name field.	A message has been displayed stating that 'Plan name required' in beside of the name field.	Pass	
3.	Enter the submit button after enter on name and without entering Description for the Description field.	A message should be displayed stating that 'Description is required' in beside of Description field.	A message has been displayed stating that 'Description is required' in beside of Description field.	Pass	
4.	Enter the submit button after entering valid a plan name and a description	A message should be displayed stating that Plan created successfully	A message should be displayed stating that plan created successfully	Pass	
5.	After Registration navigate to view plans screen	Registered plan is displayed as cards contain fields plan name, price, duration, description and two buttons for Edit and Delete	Registered plan is displayed as cards contain fields plan name, price, duration, description and two buttons for Edit and Delete	Pass	

6.	After clicking the Edit button on view plan screen	The particular plan details is visible on another screen providing option for edit the details and a Update button is present	The particular plan details is visible on another screen providing option for edit the details and a Update button is present	Pass	
7.	After clicking the Delete button on view plan screen	The particular plan details is removed and a alert message is showed.	The particular plan details is removed and a alert message is showed.	Pass	

6. CONCLUSION

The rapid growth of digital event management has created a strong need for efficient, centralized, and user-friendly platforms to organize and manage fests and institutional events. Traditional event management methods often face challenges such as manual registrations, poor coordination, lack of real-time updates, and difficulty in tracking participants and activities. Festivo is designed to overcome these limitations by providing a comprehensive web-based platform that streamlines event management through automation, structured coordination, and real-time monitoring.

A key advantage of Festivo is its centralized institution-based event management system, which allows institutions to create, manage, and monitor events through a single platform. The system simplifies participant registration, coordinator management, subscription handling, and reporting processes, making event organization more efficient and transparent. With multiple stakeholders Admin, Institution, Participant, and Coordinator the platform ensures secure role-based access control and smooth communication between all users.

By integrating modern web technologies, Festivo improves operational efficiency, reduces manual errors, enhances user engagement, and simplifies administrative tasks. The system not only benefits institutions by providing better control over event planning and execution but also offers participants a seamless and organized experience. With its scalable, secure, and user-friendly architecture, Festivo has the potential to modernize institutional fest management, making event organization more structured, efficient, and accessible.

7. REFERENCES

[1] K K Aggarwal, Yogesh Singh – Software Engineering Third Edition
[2] Roger S Presssman-Software Engineering: A Practitioner's Approach, Sixth Edition, MCGraw-Hill Higher Education
[3] Pamkaj Jalote-An Integerated approach to Software Engineerin, Second Edition ,Narosa Publishing Company
[4] https://www.stackoverflow.com
[5] https://www.sqlsolutions.com
[6] http://ecomputernotes.com/software-engineering/feasibilitystudy
[7] https://en.wikipedia.org/wiki/Third_normal_form
[8] https://www.w3resourse.com
[9] https://www.guru99.com/user-acceptance-testing.html
[10] https://www.tutorialspoint.com/system_analysis_and_design.html
[11] http://softwaretestingfundamentals.com/test-case/
[12] http://softwaretestingfundamentals.com/black-box-testing/
[13] https://www.geeksforgeeks.org/php-coding-standards/
[14] https://www.geeksforgeeks.org/software-engineering-sdlc-v-model/
[15] https://www.tutorialspoint.com/sdlc/sdlc_quick_guide.htm
[16] http://www.coddletech.com/php
[17] http://www.datatreesystems.in/technologies/mysql/
[18] http://www.wampserver.com/en/
[19] https://en.wikipedia.org/wiki/Adobe_Dreamweaver
[20] https://en.wikipedia.org/wiki/Microsoft_Word
[21] https://Draw.io

8. APPENDIX

8.1 SCREENSHOTS

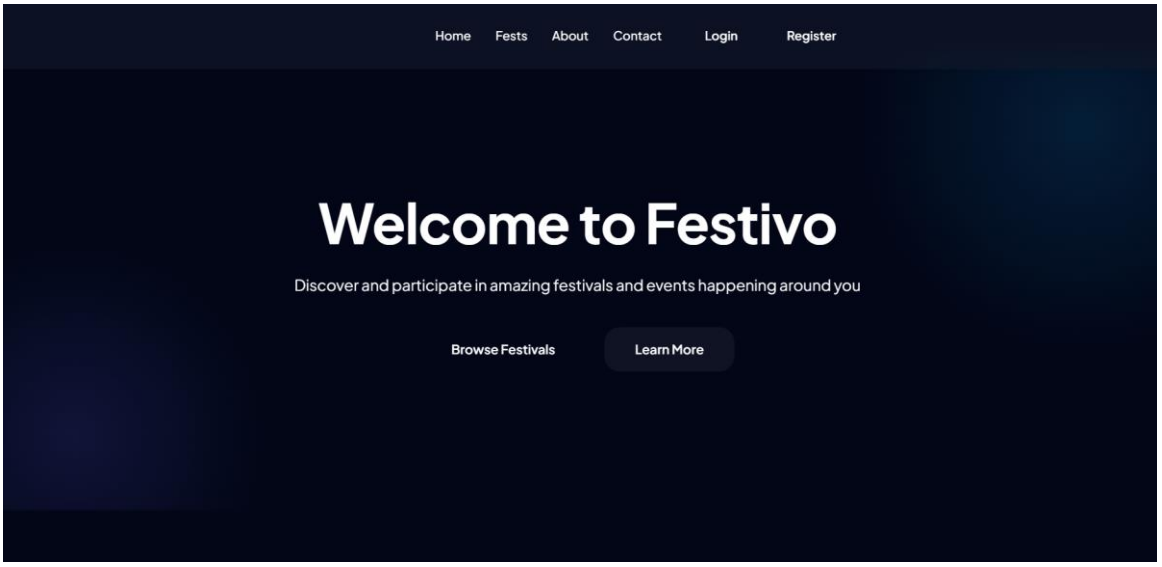


Fig 8.1 Guest Page

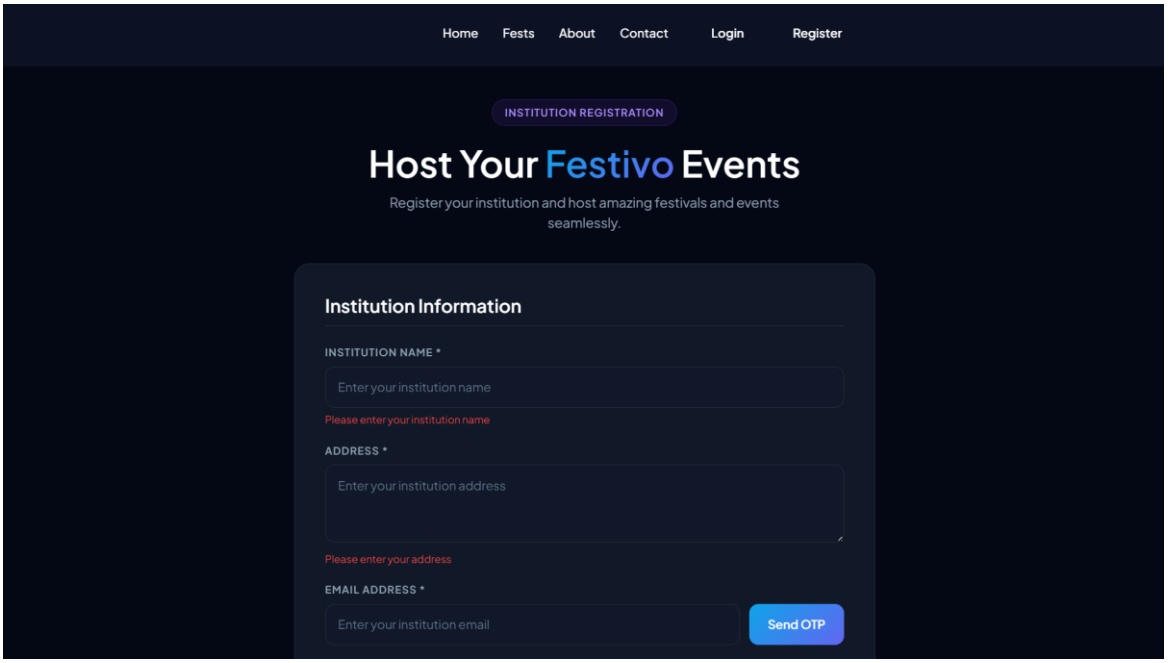


Fig 8.2 Institution Registration

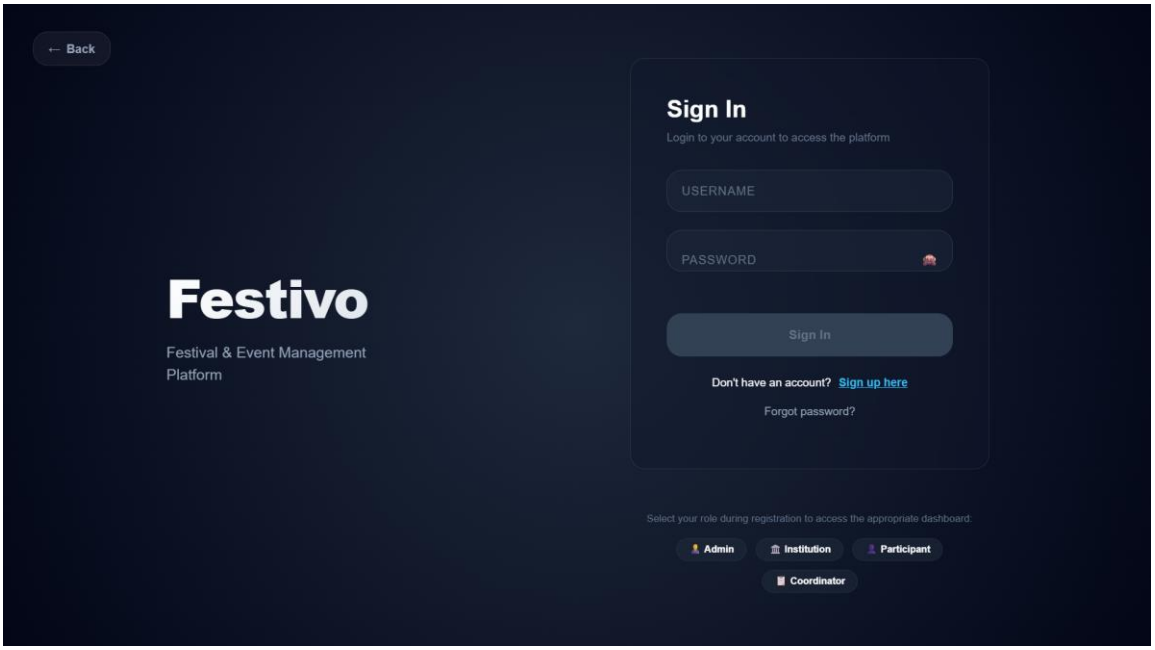


Fig 8.3 Login Page

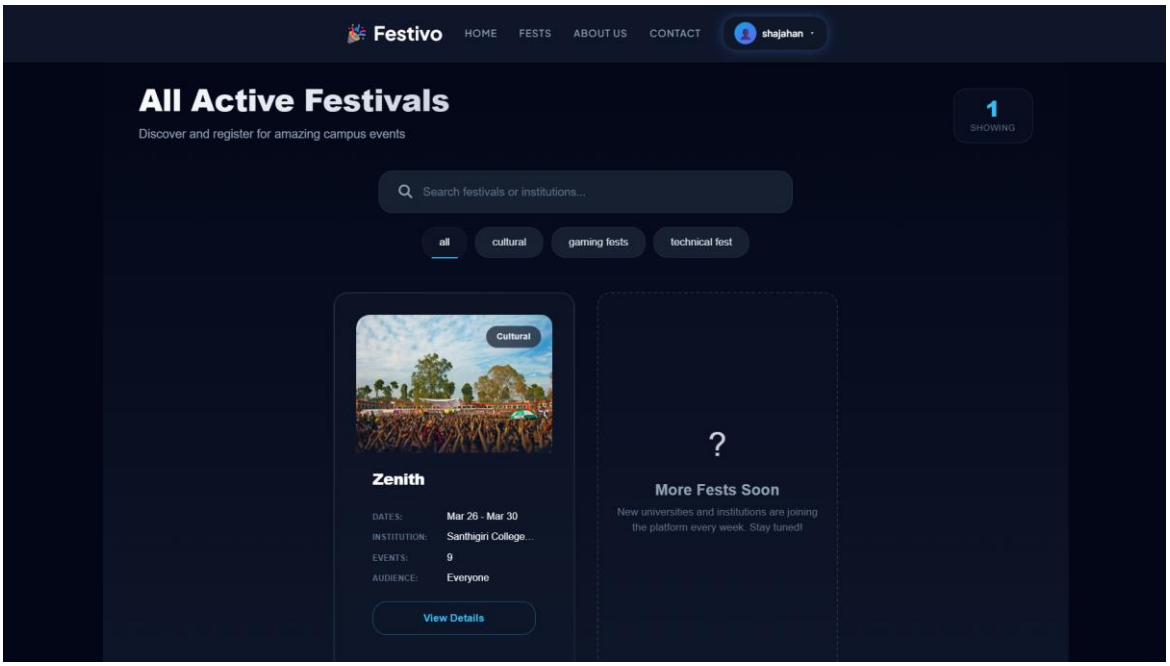


Fig 8.4 Fest view page

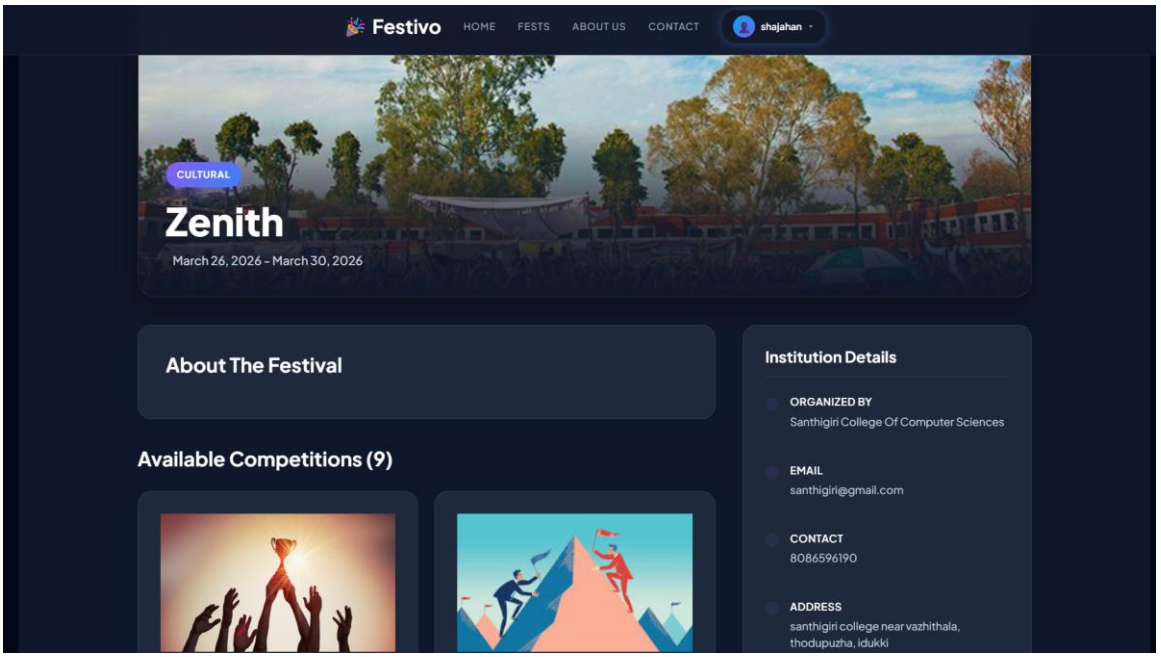


Fig 8.5 Fest details page

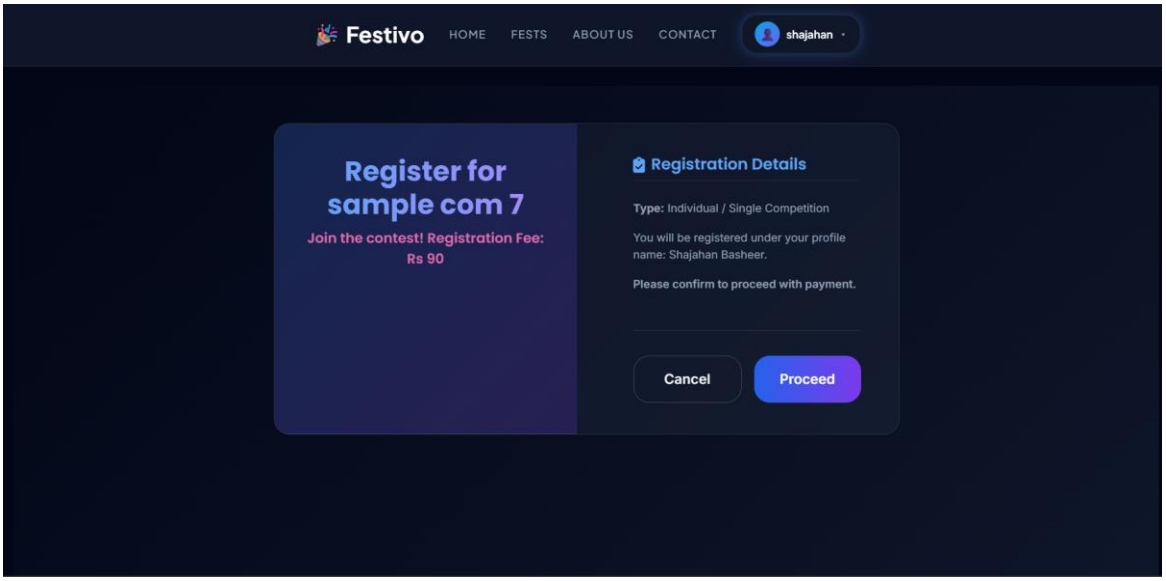


Fig 8.6 Competition registration page

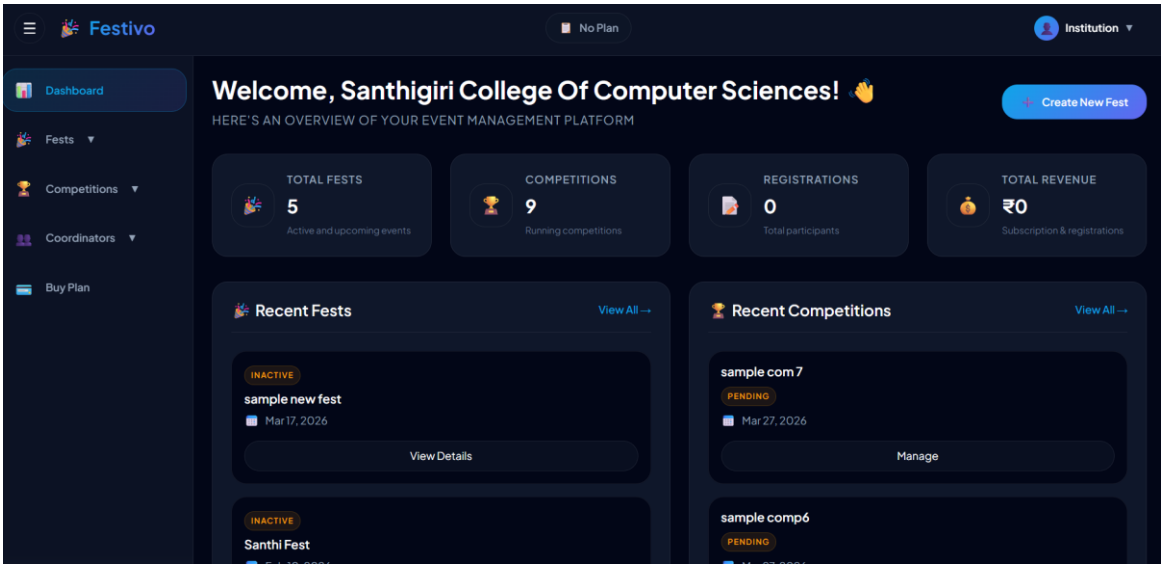


Fig 8.7 Institution dashboard

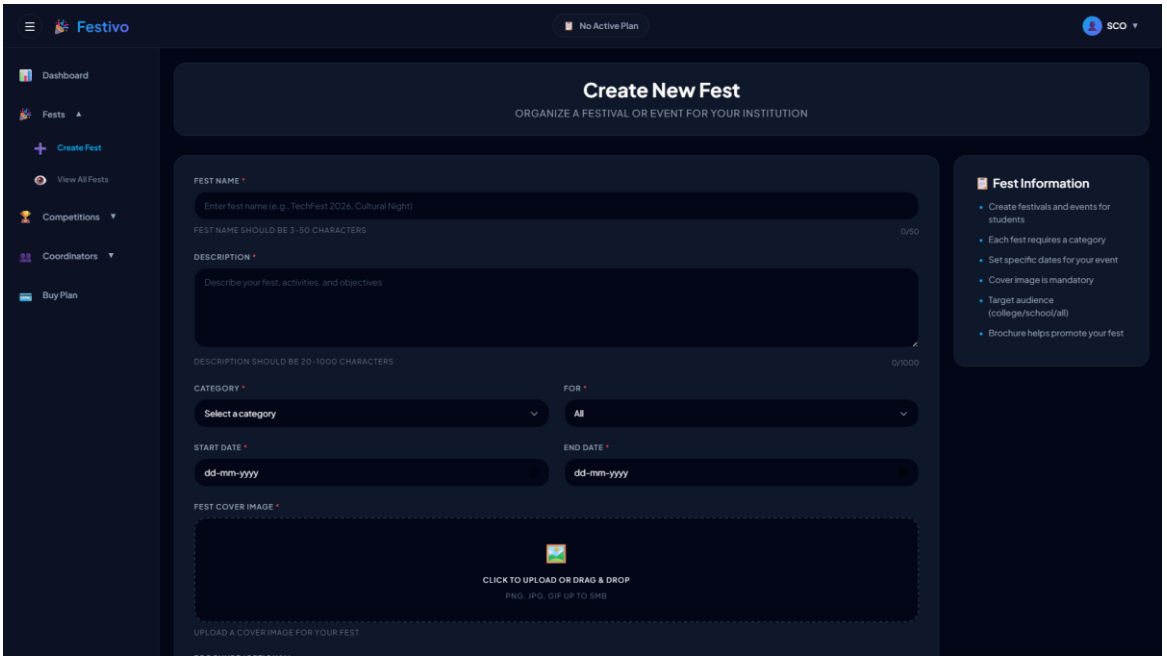


Fig 8.8 Fest creating page

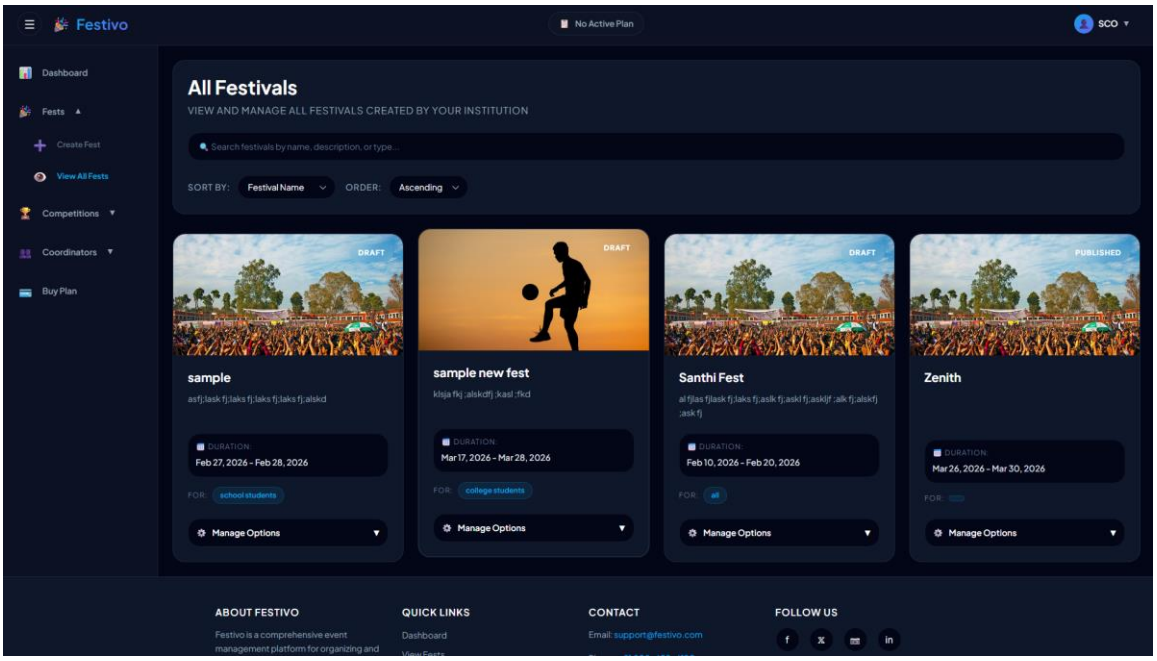


Fig 8.9 Fest View page

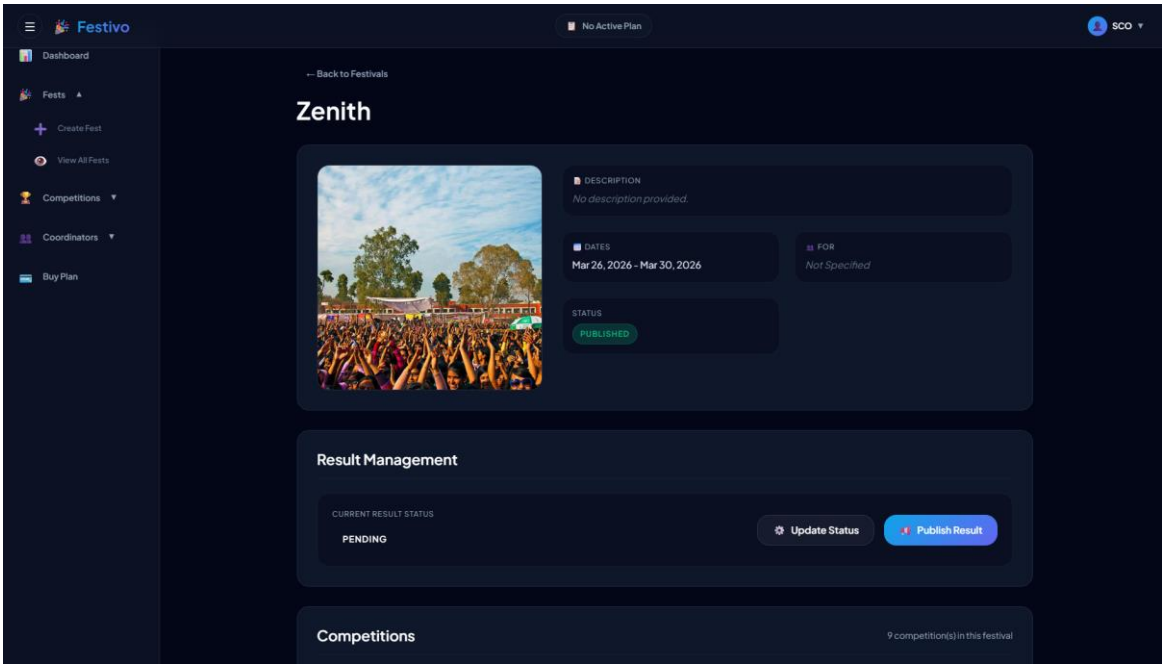


Fig 8.10 Fest details page

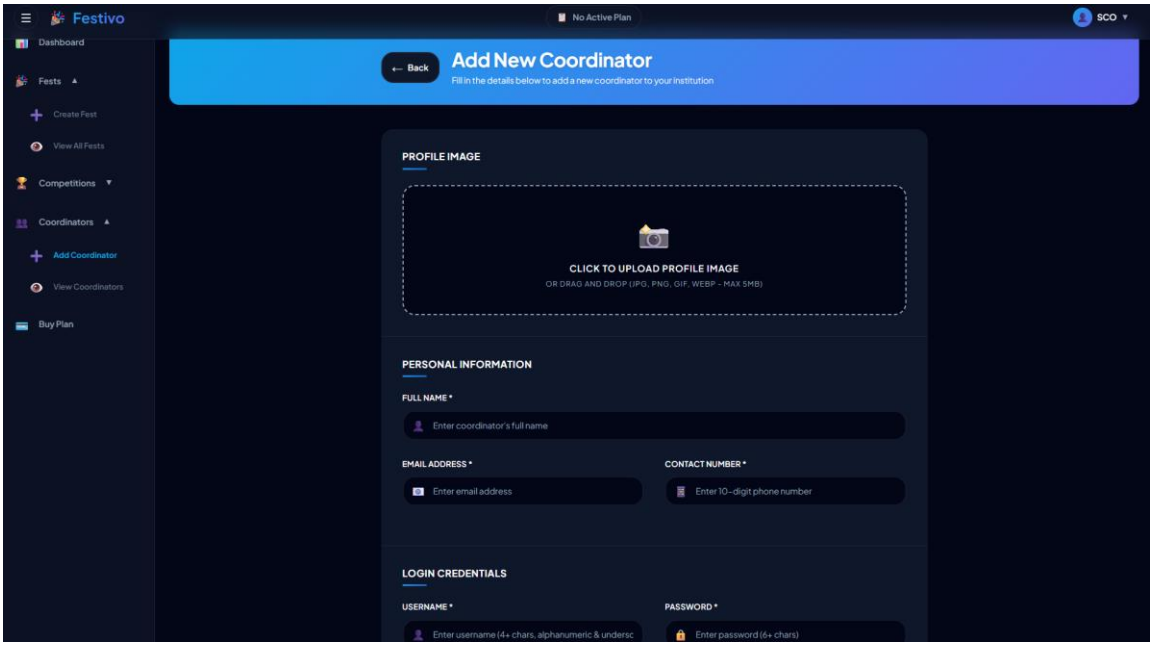


Fig 8.12 Coordinator registration page.

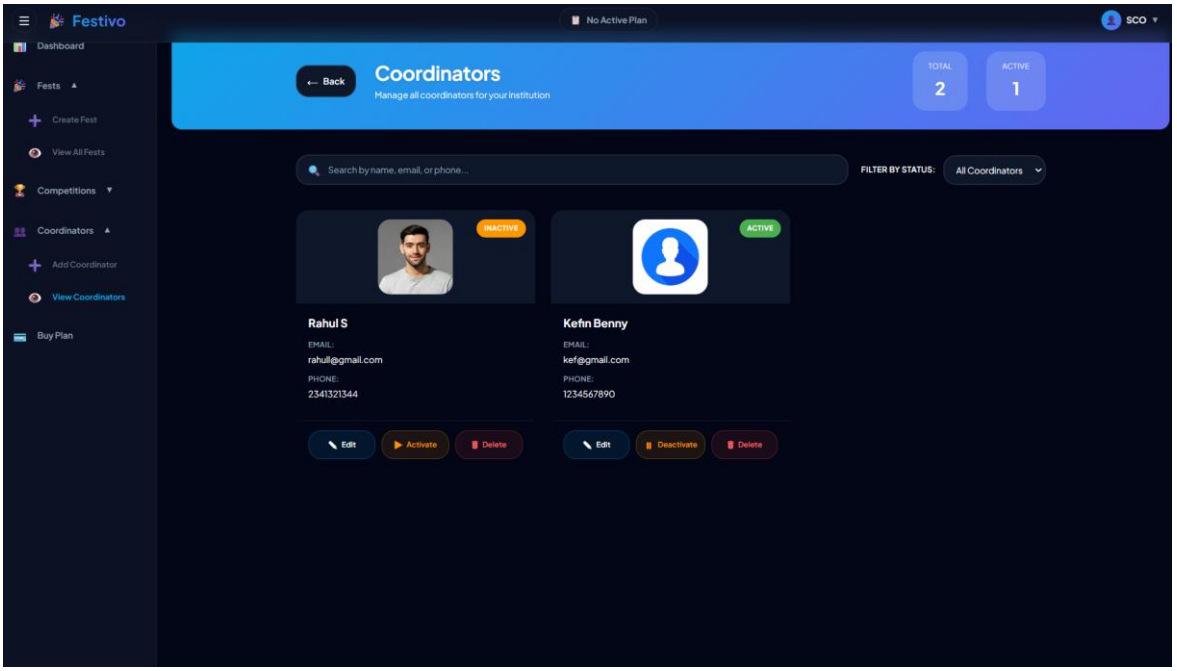


Fig 8.13 Coordinator view page

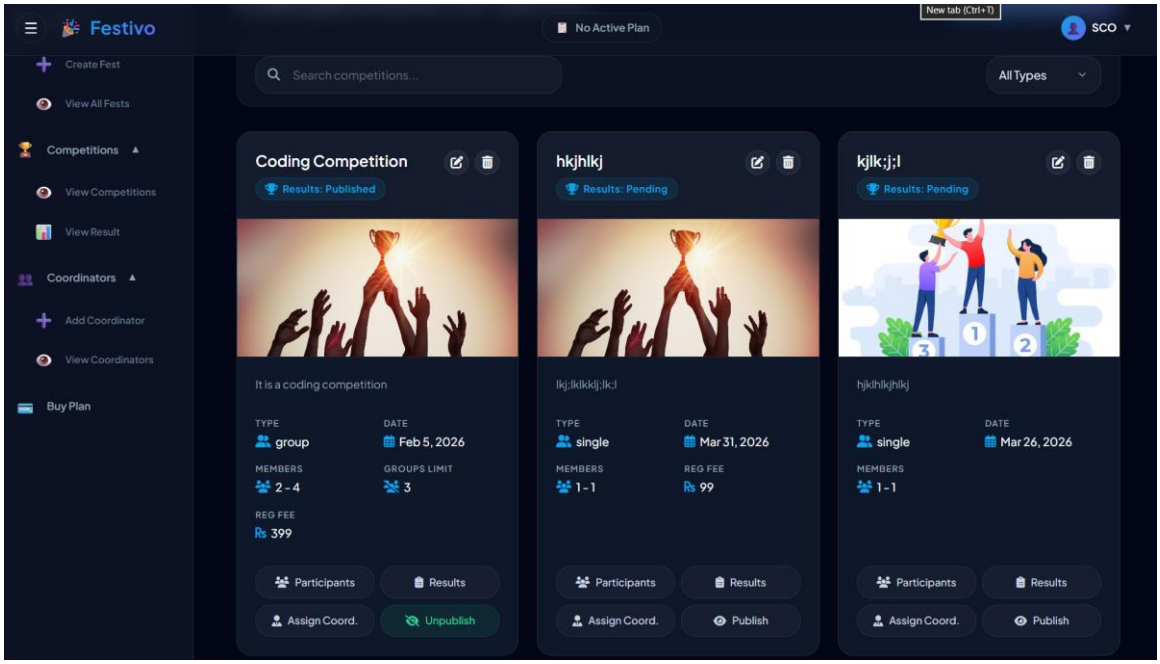


Fig 8.14 View competitions page

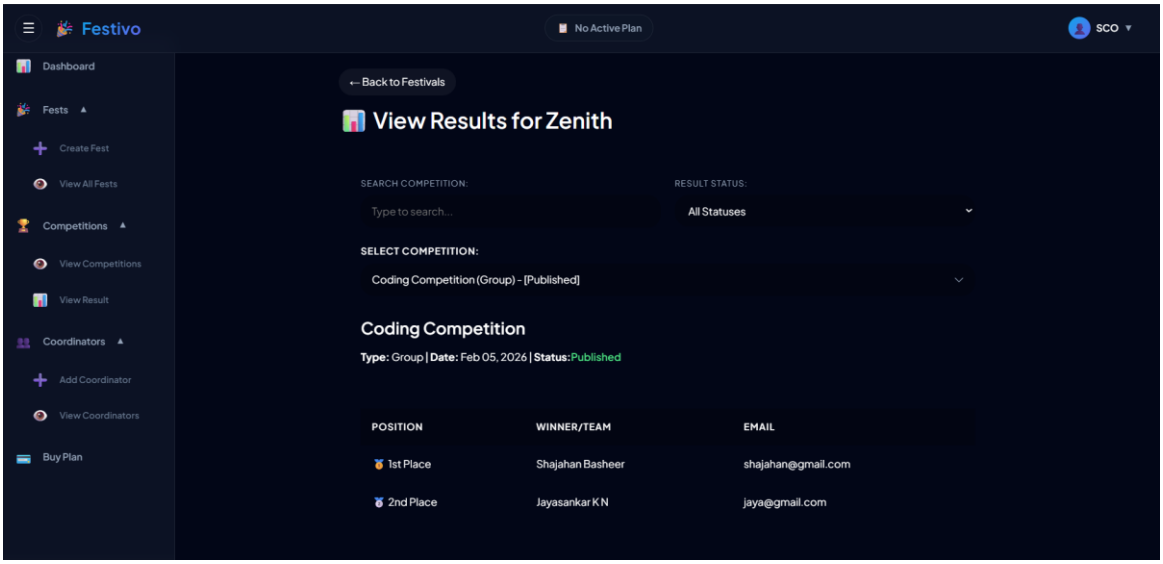


Fig 8.19 View prize page

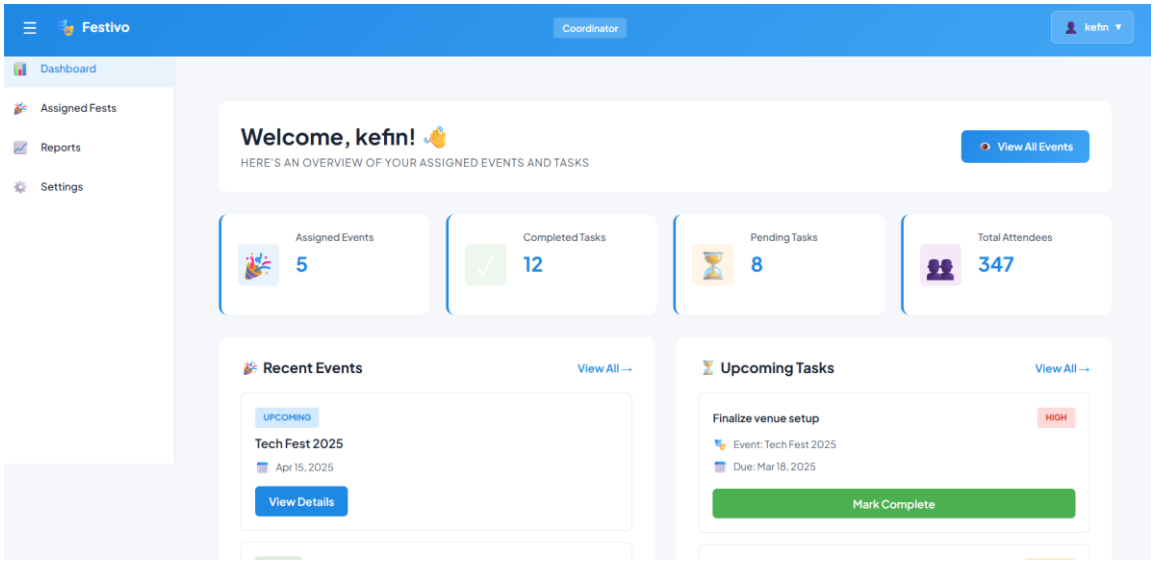


Fig 8.20 Coordinator Dashboard

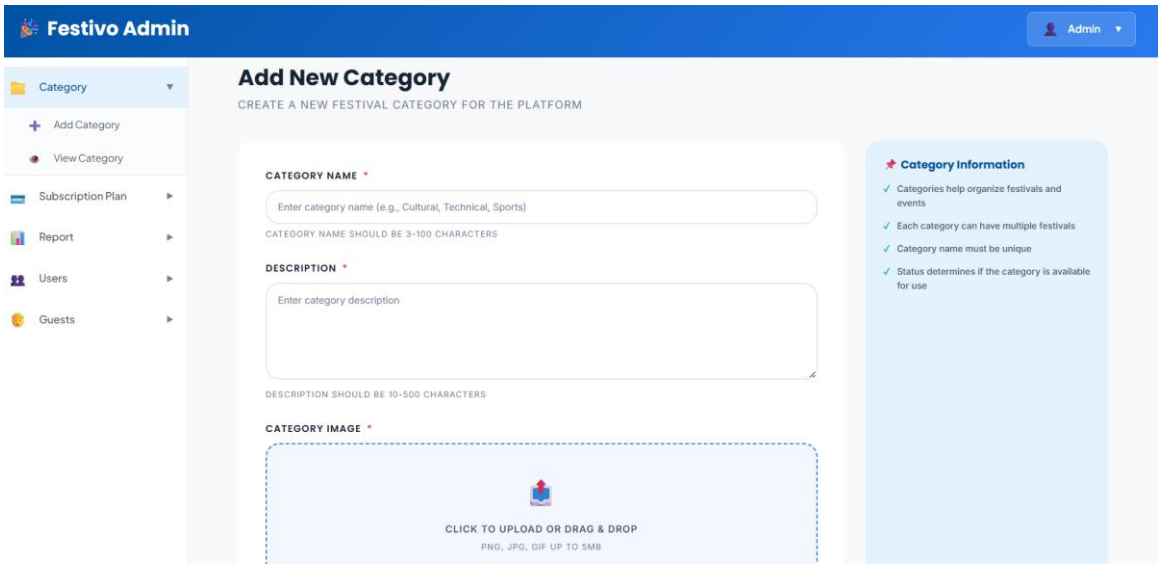


Fig 8.24 Admin category registration

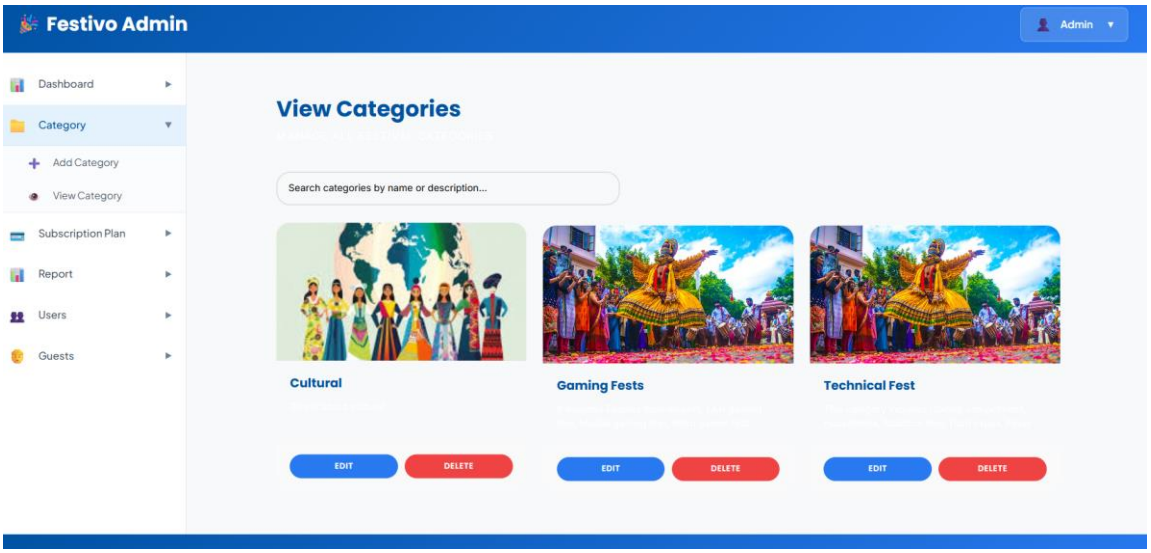


Fig 8.25 Admin category view page

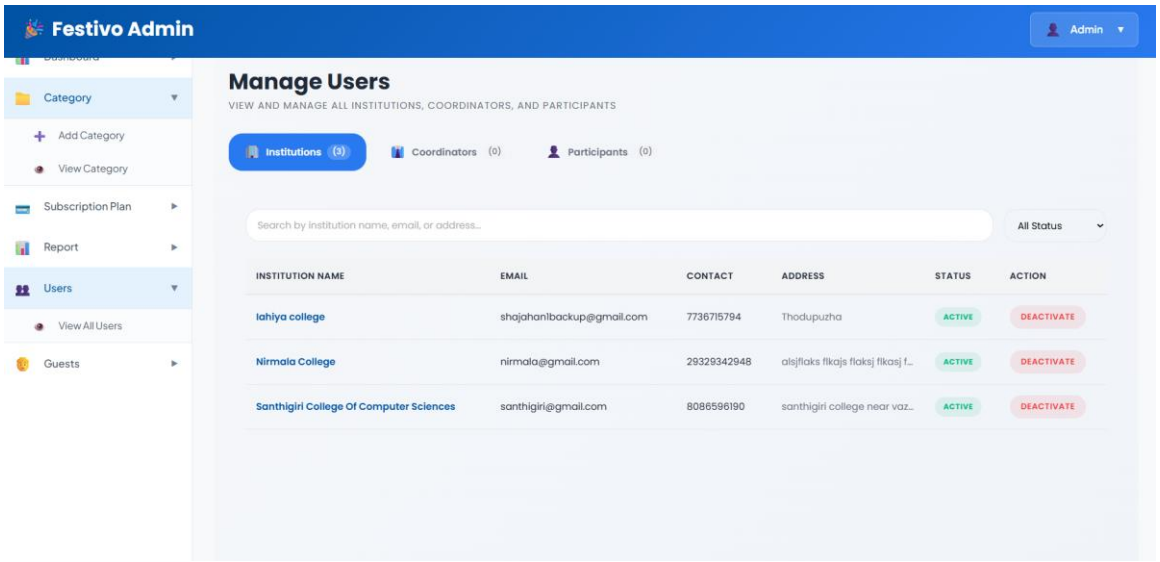


Fig 8.26 Admin view all user page